



The  
University  
Of  
Sheffield.

## Biomedical Engineering

### Final Project Report Cover Sheet

Surname: \_\_\_\_\_ Lew \_\_\_\_\_ Forenames: \_\_\_\_\_ Ja Ying \_\_\_\_\_

Registration number \_\_\_\_\_ 230167106 \_\_\_\_\_

Module code: \_\_\_\_\_ CMB344 \_\_\_\_\_

Module name: \_\_\_\_\_ Bioengineering Research Project \_\_\_\_\_

Assignment title: \_\_\_\_\_ Templating in Functional Languages \_\_\_\_\_

Supervisor: \_\_\_\_\_ Gergely Buday \_\_\_\_\_

Date submitted: \_\_\_\_\_

#### Student Declaration

I confirm that the work I have submitted is my own original work and that I have made appropriate references to any sources used.

I agree to any submitted materials being used by the Department and/or University to detect and take action on any unfair means.

I confirm that the printed version of this assessment and any electronic copy I am required to submit to TurnItIn are identical.

I understand that failure to meet these guidelines may result in one or more of the following penalties being imposed:

1. A reduced mark (including a mark of zero) being recorded for this work.
2. A reduced mark (including a mark of zero) being recorded for the unit/module.
3. A formal reprimand (warning) being placed on my student record at Departmental and/or University level which may be taken into account whenever references are written.
4. Reporting of the incident to the University Disciplinary Committee.
5. Suspension or expulsion from the University.

Student's signature: \_\_\_\_\_ JYL \_\_\_\_\_ Date: \_\_\_\_\_ 18/05/2026 \_\_\_\_\_



The  
University  
Of  
Sheffield.

## Biomedical Engineering

**Biomedical Engineering**

**Final Year Project**

# **Templating in Standard ML**

Ja Ying Lew

May 2026

Supervisor: Gergely Buday

Dissertation submitted to the University of Sheffield in partial fulfilment  
of the requirements for the degree of

*(Bachelor of Engineering In Biomedical Engineering)*

## **Abstract**

Mustache is a logic-less templating language used widely in web development. Its multi-language design enforces a strict separation between templates and program logic. Template engines should not compute anything, only substitute values and iterate over data. This makes templates safe and easy to reason about, but it creates a problem. Any logic the template needs, whether an item is first in a list, whether a description is needed, or whether something is empty, has to be precomputed or specified after checking the output. As a result, this means someone has to manually add this in, which defeats the point of automation.

This dissertation proposes a pre-processing framework designed to enrich input data with automated contextual inputs. By injecting structural and positional information before the data reaches the rendering engine, this solution enables dynamic output variations while preserving existing template syntax and the inherent advantages of template-based systems.

The implementation utilises parser generation to construct an abstract syntax tree (AST). The system comprises of rendering system compatible with logic-less templating engines and a pre-processor that automates the injection of structural context. By leveraging a strong type system, the architecture provides compile-time verification to ensure all data transformations are validated before rendering begins.

The prototype is tested with 47 unit tests across three suites covering the parser, preprocessor, and renderer, all passing on Standard ML compiler. While current limitations include incomplete support for partials and delimiter changes, this work demonstrates the potential for making logic-less templating context-aware while maintaining its fundamental safety.

## Acknowledgements

Time flies so fast, and I can't believe that my university chapter is ending. My degree and university experience have been an absolute thriller rollercoaster and have taught me wonders and I am deeply grateful for the growth I have experienced over these past few years.

I would like to express my sincere gratitude to my supervisor, Gergely Buday (future Dr. Buday). Without the guidance of my supervisor, I would not have reached this point in my project or my experience in computer science. I am forever grateful for his support, guidance and regular meetings we have had. It was an honour, and his wisdom and patience truly inspire me.

Also, I would like to mention my course mates, Priscille, Ameerah, Deepti and Zahra for gracing me with their kindness and intellect, always pushing me to my best. Without them, I truly would not be in the position I am in now and biomedical engineering would be far less amusing.

I would also want to mention to all the study buddies through this journey: Tanya, Lipxin, Brandon, Sanjith, Shreyas, Sol, Isaac, Sarwin, Anson and so many others. I am equally grateful for all my flatmates, Juli, and someone special called Cheng? To everyone who has been part of this journey, it has been a pleasure.

Last but not least, I would like to take a moment of gratitude for my family, especially my Mum, for their immense support and always cheering me on. I am truly blessed to have such wonderful people in my life- thank you.

# Contents

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                    | <b>1</b>  |
| <b>Acknowledgements</b>                                            | <b>1</b>  |
| <b>List of Figures</b>                                             | <b>5</b>  |
| <b>1 Introduction</b>                                              | <b>1</b>  |
| 1.1 Functional Languages Usage in Biomedical Engineering . . . . . | 4         |
| <b>2 Literature Review</b>                                         | <b>5</b>  |
| 2.1 Template theory and Systems Design . . . . .                   | 5         |
| 2.2 Template Landscapes . . . . .                                  | 7         |
| 2.2.1 Index 1: Logic-less systems . . . . .                        | 7         |
| 2.2.2 Index 2: Logic Friendly with Sandboxing . . . . .            | 8         |
| 2.2.3 Index 5: . . . . .                                           | 8         |
| 2.2.4 Logic Friendly without Sandboxing . . . . .                  | 9         |
| 2.3 Functional Languages for Template Processing . . . . .         | 10        |
| 2.3.1 Haskell: Type-checked Metaprogramming . . . . .              | 10        |
| 2.3.2 OCaml: Industrial Pragmatism . . . . .                       | 10        |
| 2.3.3 Elm in Elm-mustache: Closest to prior art . . . . .          | 11        |
| 2.3.4 Standard ML . . . . .                                        | 11        |
| 2.3.5 Takeaway . . . . .                                           | 12        |
| 2.4 Parsing Theory and Strategies . . . . .                        | 12        |
| 2.5 Parser Generators: ML-Lex and ML-Yacc . . . . .                | 14        |
| <b>3 Requirements and Analysis</b>                                 | <b>18</b> |
| 3.1 Problem Statement . . . . .                                    | 18        |
| 3.2 The Task: . . . . .                                            | 20        |
| 3.3 Requirements . . . . .                                         | 20        |
| 3.4 Breakdown . . . . .                                            | 21        |
| 3.5 Testing and Evaluation . . . . .                               | 21        |
| 3.6 Legal and Ethical Considerations . . . . .                     | 23        |

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>4</b> | <b>Design</b>                                                | <b>24</b> |
| 4.1      | Choice of Implementation Language . . . . .                  | 24        |
| 4.2      | Pipeline Choice . . . . .                                    | 25        |
| 4.3      | Parser Design . . . . .                                      | 26        |
| 4.3.1    | Choice of Parser Generator . . . . .                         | 27        |
| 4.3.2    | SML Mechanisms and Functors . . . . .                        | 27        |
| 4.3.3    | AST Shape . . . . .                                          | 27        |
| 4.4      | Preprocessor Design . . . . .                                | 28        |
| 4.4.1    | A Worked Example . . . . .                                   | 29        |
| 4.5      | Renderer Design . . . . .                                    | 29        |
| 4.5.1    | Truthiness . . . . .                                         | 30        |
| 4.5.2    | Dot-Separated Path Resolution . . . . .                      | 30        |
| 4.6      | Build and Deployment . . . . .                               | 30        |
| <b>5</b> | <b>Implementation and Testing</b>                            | <b>32</b> |
| 5.1      | Implementation Overview . . . . .                            | 32        |
| 5.2      | Implementing the Parser . . . . .                            | 32        |
| 5.3      | Implementing the Preprocessor . . . . .                      | 33        |
| 5.4      | Implementing the Renderer . . . . .                          | 34        |
| 5.5      | Coding Issues Encountered . . . . .                          | 34        |
| 5.6      | Testing Strategy . . . . .                                   | 35        |
| 5.7      | Test Results . . . . .                                       | 35        |
| 5.7.1    | Specification testing . . . . .                              | 36        |
| 5.8      | Limitations Confirmed by Testing . . . . .                   | 37        |
| <b>6</b> | <b>Results and Discussion</b>                                | <b>38</b> |
| 6.1      | Findings and Systems Evaluation . . . . .                    | 38        |
| 6.1.1    | Functional Prototype . . . . .                               | 38        |
| 6.1.2    | Testing Evaluation . . . . .                                 | 38        |
| 6.2      | Evaluation of Project Goals . . . . .                        | 39        |
| 6.3      | Comparative Analysis and Further work . . . . .              | 39        |
| 6.4      | Further work . . . . .                                       | 40        |
| <b>7</b> | <b>Conclusions</b>                                           | <b>41</b> |
| <b>A</b> | <b>Appendix</b>                                              | <b>43</b> |
| <b>B</b> | <b>Mustache Template Example: Student Profile Generation</b> | <b>44</b> |
| <b>C</b> | <b>Rendered Output</b>                                       | <b>47</b> |
| C.1      | "small_test" rendered output . . . . .                       | 47        |

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| C.2      | "animals" rendered output snippet . . . . .         | 47        |
| C.3      | "users_dataclass" rendered output snippet . . . . . | 49        |
| <b>D</b> | <b>Project Management</b>                           | <b>53</b> |
| <b>E</b> | <b>Self-review</b>                                  | <b>55</b> |

# List of Figures

|     |                                                                                                                                                                                                                                                                                                                                                                                                                       |    |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | A snippet template (left) and the email it produces when filled in with one customer's data (right). The double braces mark the slots where information is inserted from a dataset . . . . .                                                                                                                                                                                                                          | 1  |
| 2.1 | Model-View-Controller State and Message Sending (adapted from Krasner & Pope, 1988) . . . . .                                                                                                                                                                                                                                                                                                                         | 6  |
| 2.2 | Mapping of compiler phases to the "Chomsky hierarchy". The diagram illustrates the increasing levels of computational complexity required for early-stage analysis: from Regex pattern-matching tokens (Type-3) to validating hierarchical structure (Type-2) and enforcing contextual logic (Type-1). In reality, some phases may be combined. Adapted from [48]. . . . .                                            | 13 |
| 3.1 | The visual workflow before this project. A raw dataset (a) is hand-organised into JSON and passed through a logic-less Mustache template (b) to produce a simple README (c) . . . . .                                                                                                                                                                                                                                 | 18 |
| 3.2 | <b>Manual preprocessing (left) versus the proposed automated preprocessor (right).</b> The manual route introduces maintenance overhead, no verification mechanism, and silent failures detectable only by inspection; the preprocessor removes the hand-written step and converts an entire class of omission error into a compile-time error, while the Mustache renderer remains unmodified in both cases. . . . . | 19 |
| 4.1 | The preprocessing pipeline. The Mustache template is parsed into an abstract syntax tree (AST); the JSON data is parsed into a raw JSON value; the preprocessor walks the raw JSON and emits an enriched JSON value containing the contextual flags the template requires; the renderer consumes both the AST and the enriched JSON to produce the final output string. . . . .                                       | 24 |
| D.1 | Gantt chart showing major phases from October 2025 to May 2026. Orange bars indicate research and writing phases; blue bars indicate implementation work. Detailed weekly breakdown provided in Sections 2.3 and 2.4. The critical path runs through pattern matching learning (Feb) and core implementation (Feb-Mar). . . . .                                                                                       | 53 |



# 1. Introduction

Templating engines are one of the most common operations in modern software. Commonly used in web development, template engines are systems that generate outputs such as emails or documents from data files like JSON (JavaScript Object Notation). Any tool designed to generate source code from a specification will, at some point, separate a fixed structural skeleton from the variable data used to populate it. This separation is what a template system provides, enabling mass automation of tailored outputs. A practical example of this is a company that automates SDK (Software Development Kit) generation for its clients, since each client is different. Another example can be shown in 1.1, the left shows what the template would look like for the developer, and the right shows the generated output, which gets delivered to the customer, 'Sarah'.

| Template                    | Rendered email                                |
|-----------------------------|-----------------------------------------------|
| Dear {{name}},              | <i>Dear Sarah,</i>                            |
| Thank you for your order.   | <i>Thank you for your order. Your copy of</i> |
| Your copy of {{book_title}} | <i>Wuthering Heights will be delivered on</i> |
| will be delivered on        | <i>14 May.</i>                                |
| {{delivery_date}}.          | <i>Best wishes,</i>                           |
| Best wishes,                | <i>The Bookshop</i>                           |
| The Bookshop                |                                               |

*Figure 1.1: A snippet template (left) and the email it produces when filled in with one customer's data (right). The double braces mark the slots where information is inserted from a dataset*

Templating is embedded in our everyday lives: from web pages, emails, documents, and code generation. While famously used for web development, templating engines have expanded far beyond web development to DevOps and CI/CD pipelines [1], infrastructure as code [2], network automation [3], and general software tasks [4] and more.

Despite the significance of templating engines, there is a fair security issue that exists with them. Server-side template injections (SSTI) is a vulnerability which arises when user-supplied input is embedded into a template without being treated as protected data, allowing the attacker to inject executable code into the template engine [5]. While many developers confuse these flaws for cross-site scripting (XSS), which is an attack on the browser, exploiting vulnerabilities in client-side code, SSTI is more dangerous [6]. Being a risk highlighted by Open Worldwide Application Security Project (OWASP), SSTI is an attack on the server,

meaning the attacker may gain access to the server's file system and environment [7], [8]. The canonical study of the problem by Kettle [9] demonstrates that this typically can escalate to remote code execution (RCE), granting attackers full control over servers or applications. The real-world impact of SSTI is highlighted by the high-profile breach of uber in 2016, in which Orange Tsai exploited Flask's Jinja2 engine on 'rider.uber.com' by simply setting a profile name to a mathematical expression within a template tag, specifically `''7'*7'`, which the Uber mail server evaluated and returned as `'7777777'` in the confirmation email, earning him a \$10,000 bug bounty for what was ultimately full remote code execution [10]. SSTI is not confined to one engine: documented exploits exist for Jinja2, Twig, Smarty, FreeMarker, Velocity, Thymeleaf, and Jade, among others, with many of the underlying engines downloaded millions of times a month [5], [11]. OWASP has listed SSTI vulnerabilities are among the most critical risks to the web applications for over a decade [7].

An overview of the security vulnerabilities of templating engines is discussed. The systematic analysis of 34 popular template engines across 8 programming languages found that 31/34 allowed RCE, while fewer than a third implement any built-in security protection, such as sandboxing or restricted execution [5]. Even where defences exist, they are routinely bypassed: a 2023 study of 7 widely-used PHP template engines identified 135 new template-escape bugs and synthesised working RCE exploits against 55 of them [12]. The vulnerability is also not a rarity in practice: it accounted for a measurable share of critical-risk findings in a commercial web application audit [5]. Remediation tends to be slow, extending the window during which affected applications remain exposed.

In response to these security issues is to take away the template's ability to execute code or involve business logic. Thus, enforcing a strict separation between presentation and application logic was argued for [13] on grounds of maintainability and security. A need for a template that cannot compute or be tricked into computing something the developer did not intend.

Mustache released by Chris Wanstrath in 2009, introduces a 'logic-less' framework to separate business logic from presentation through reducing the available operations to variable substitution, section iteration and no conditionals or loops [14]. Because the syntax is so restricted, the same Mustache template can be rendered by an independent implementations in more than 40 programming languages, including Ruby, JavaScript, Java, PHP, Haskell, and OCaml [15], [16]. Mustache takes data in the format of a JSON file, and uses tags in desired implementation language like `{{#ifString}}` to call on and then renders the desired output; an example is provided in the Appendix for clarification B.

Specifically, the price of this safety is what the literature calls the *preprocessing burden*. Anything that the template would have computed itself in a logic-friendly system, the developer must now compute in advance and embed in the data. If a list needs commas between

items but not after the last one, the developer has to walk the list manually and attach an 'is-Last' flag to each element. If a field might be missing, the developer has to add a companion 'has\_field' flag to make the template's conditional render without missing it. None of this is difficult, but all of it is tedious, error-prone, and untyped. Nothing checks that the developer remembered to add every flag, and a missing flag is silent failure at compile time and only visible by inspecting the rendered output.

Logic-less templates are 'dumb' enough to evade these security issues; however, they have argued to have a trade-off. Logic-friendly templates are convenient but expose applications to security attacks, whereas logic-less templates are safe but divide business logic from presentation logic of the template. Critics [17], [18] have made a similar point of 'the logic has to go somewhere' [19]: instead of a logic-friendly template engine, the developer has to now precompute the 'logic' and embed the data into the template. The cost of preprocessing has two components, and the relationship between them matters more than either in isolation. The runtime cost is small. Research on type-based security in templating frameworks illustrates how dramatically architecture determines runtime cost: Samuel et al. [20] report that compile-time, type-based approaches to context-sensitive sanitisation incur overheads of 3-10%, against 78% to over 500% for purely runtime approaches measuring the same property. Preprocessing JSON to add contextual flags is a similarly bounded, compile-time-friendly operation. The cost that actually persists, however, is the developer's. Every new template, every change to the underlying data model, and every newly-added field potentially introduces a fresh set of flags that must be identified, computed, and embedded into the rendering context by hand. A list that gains a new sorting requirement needs a new flag; a field that becomes optional needs a companion 'has\_field' flag added wherever the template references it; a section that becomes nested needs position flags propagated through the structure. Nothing in the toolchain checks that this work has been done: a missing flag produces a stray comma or a malformed sentence in the rendered output, with no error message and no compile-time warning. The preprocessing burden of logic-less templating is therefore not primarily a performance problem but a correctness and verification problem that recurs across the lifetime of the application, and it is this cost that the present work addresses.

This dissertation investigates whether Standard ML, a functional language, can automate that preprocessing while preserving Mustache's logic-less safety benefits. Functional programming properties, such as algebraic data types for tree-shaped data, exhaustive pattern matching, and robust static typing, are as effective for preprocessing automation as they are for traditional parsing and transformation tasks. The contribution of this project is a working prototype. It consists of a Mustache parser, a JSON preprocessor, and a renderer that consumes the enriched data. The preprocessor sits in front of any standard Mustache implementation, requires no changes to the Mustache specification, and uses Standard ML's pattern-matching exhaustiveness check to ensure that every transformation is applied consis-

tently across nested data structures. All 47 unit tests across the three components pass, and the tool runs end-to-end from the command line.

Chapter 2 reviews the literature on template systems, examining alternative engines across platforms and investigating the parsing theory that underpins their implementation. Chapter 3 defines the preprocessor requirements, breaks the problem into manageable components, and establishes the evaluation criteria. Chapter 4 justifies the major design decisions: language choice, architectural approach, and implementation strategy, against the alternatives considered. Chapter 5 describes the implementation, documents the technical challenges encountered, and presents the testing methodology and results. Chapter 6 evaluates the prototype against its original aims, compares the typed preprocessing approach to manual alternatives, and identifies directions for further work.

## **1.1 Functional Languages Usage in Biomedical Engineering**

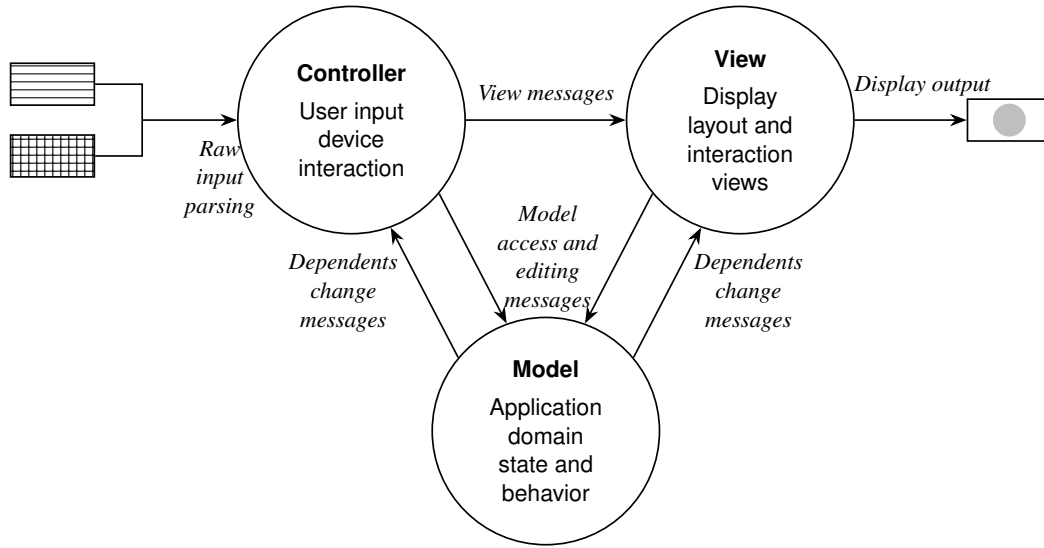
Functional languages have a long history in scientific computing, and BioCaml [21] is the most visible example of their adoption in biology: a comprehensive OCaml library for sequence analysis, file format parsing, and high-performance genomics. The case for typed functional languages in bioinformatics has been argued in detail by Berenger *et al.* [22], who identify the same properties this dissertation relies on. Strong static typing and Hindley-Milner inference allow complex hierarchical data structures to be represented precisely and checked for consistency at compile time, while algebraic data types and pattern matching let programs follow the shape of the data, and the compiler's exhaustiveness check catches missing cases before a long-running analysis crashes on input it was not prepared for. The preprocessor described in this dissertation uses an algebraic data type to represent the parsed Mustache template and exhaustive pattern matching to guarantee that every contextual flag a template requires has a corresponding case in the preprocessing function: the same engineering values, applied to a different domain.

## 2. Literature Review

### 2.1 Template theory and Systems Design

The Model-View-Controller principle, articulated by Krasner and Pope (1988) in Smalltalk-80 (an object-oriented programming language developed in the 1970s [23]), established the foundational separation of concerns that underpins modern template systems. This introduced a three-way factoring: models that encapsulate the logic of the application domain, views render graphical representations, and controllers that mediate user input, illustrated in 2.1 [24]. Now, in modern web architectures, template systems function as 'View', tasked solely with transforming model data into user-facing documents. Ideally, this should restrict modifications to presentation (views) to be strictly independent of application behaviour (models). In practice, this means separation should exist as a distinction between application logic (the model) and the presentation markup/ template code (the view), illustrated by 2.1; in between this lies the Command-Line Interface (CLI) as the controller. Template systems separate presentation markup from application code through 'separation of concerns': business 'model' logic (data processing, calculations) remains distinct from presentation 'view' logic (display structure). This separation improves maintainability, reduces security vulnerabilities, and enables parallel development by allowing designers to modify presentation independently from programmers updating business logic [13]. However, as template engines evolved, many abandoned this strict separation in favour of developer convenience, motivated by 'processing burden'. In a logic-less system, the view layer is incapable of evaluating state; it cannot determine if a collection is empty, calculate list indices or identify certain cases. Consequently, all these presentation-specific states must be explicitly computed in the 'model' data structure prior to rendering. To bypass this, engines have begun allowing conditional logic and data manipulation directly within the view layer [25]. Recent research identified 'mismatched data contexts' and 'syntax misuse' as the dominant root causes of bugs in modern templating applications, causing 19.42% and 35.67% of bugs, respectively [25]. A 'mismatched data context' occurs when raw, unprepared 'data context prepared by the host frequently mismatches the semantic expectations of the template placeholders" [25]. Furthermore, "syntax misuse' is highly prevalent because developers attempt to implement complex conditional logic using limited, uncompiled syntax of template engines; an example includes, "Handlebars does not support nesting mustache expressions" [25]. Due to the deferred validation of template engines, these data mismatches and syntax bugs only reveal themselves as silent rendering errors at runtime. This architectural limitation directly motivates the selection of Standard ML for this project. By handling the preprocessing burden within SML, its strict static compiler can catch these issues as explicit compile-time warn-

ings long before the template is rendered, a structural advantage that will be detailed further in the system design section.



*Figure 2.1: Model-View-Controller State and Message Sending (adapted from Krasner & Pope, 1988)*

To understand why the industry drifted towards these vulnerabilities, understanding the preprocessing burden imposed by logic-less templates is key. While systems like Mustache successfully enable the separation of business logic within the view layer, they do not eliminate bugs but shift the point of failure to the host language. Because a logic-less view cannot evaluate state, all presentations must be explicitly predefined. Delegating these manual tasks to the model layer necessitates creating bloated view models, which may result in an inverse separation effect, where business logic is polluted by presentation constraints. This ends up being exhaustive and tedious, but leads to being highly error-prone. Because traditional architectures lack an enforceable interface to verify that the prepared data satisfies the template’s semantic requirements, a vulnerability called, ‘Opaque Data Flow’; any human error during this manual preparation inevitably manifests as a ‘Mismatched Data Context’ rendering failure [25].

To escape the preprocessing burden, the industry drifted towards more computationally powerful templates (e.g. Jinja2), allowing developers to embed logic into the view. However, this drift fundamentally goes against the MVC architecture [13]. This review evaluates this structural failure through Parr’s (2004) framework, which defines 5 "entanglement" violations conditional logic based on view structure, method/function calls from templates, value computation within templates, model scope manipulation, and recursive rendering [13]. Parr shows that unrestricted templates inevitably violate separation when developers embed business logic, as templates end up producing logic, with template computational power classified from weakest (substitution-only) to strongest (turing-complete).

Beyond rendering bugs, this entanglement introduces the severe security vulnerabilities high-

lighted previously. Pisu et al's (2025) evaluation of 34 template engines confirms that granting templates computational power directly links to SSTI to escalate to RCE [5]. Engines permitting complex logic and object access (e.g Jinja2, EJS) are highly susceptible to these fatal exploits, even suffering documented sandbox bypasses [5]. This highlights a fundamental dilemma: strict separation (Mustache) enforces security but imposes a severe, error-prone preprocessing burden. On the other hand, unrestricted power (Jinja2) reduces preprocessing at the cost of critical vulnerabilities and entanglement [13]. Therefore, an effective template system must satisfy three key design criteria [13]:

1. It must enforce the "separation of concerns", ensuring business logic remains in the model.
2. It must provide clear syntactic boundaries that distinguish presentation structure from application data.
3. It must balance computational power appropriately, restricting it to only what is necessary for presentation while avoiding unnecessary complexity in the model layer.

Having established the theoretical requirements for strict model-view separation and the security implications of entanglement, it is necessary to examine how current industry tools navigate this architectural dilemma.

## 2.2 Template Landscapes

The variation between logic-friendly and logic-less design recurs across every major programming platform. Specifically Parr identifies Entanglement Index ranging from 1 to 5 of which template engines are usually polarised to either Index 1 (strictly separated) or Index 5 (totally entangled). Thus, it can be identified that the problem is the tension of this balance [13].

### 2.2.1 Index 1: Logic-less systems

At Index 1, these are the engines that strictly forbid logic, math, or conditionals within the template and are the most strict on separation.

- **Mustache and Handlebars:** Mustache's widespread adoption across a dozen languages demonstrates a demand for a logic-less template engine, independent of any particular language's syntax [26]. Handlebars extends Mustache by adding helpers, block expressions, and partials, compiling templates into JavaScript functions for faster execution [27]. However, Handlebars can be contradictory. Helper proliferation is when developers write dozens of custom helper functions to perform basic operations (comparisons, arithmetic, formatting), that the logic-less syntax prohibits, ef-

fectively rebuilding a programming language through helpers while claiming the templates remain "logic-less."

- **Liquid:** Developed by Shopify, it was designed to specifically to allow untrusted third-party authors to be able to customise e-commerce themes without compromising the server [28]. This is aimed to be secure, and each theme does not directly impact the others.

### 2.2.2 Index 2: Logic Friendly with Sandboxing

At Index 2, these engines allow view logic (like loops and conditionals) but attempt to contain it within a restricted environment to prevent system access and reduce preprocessing burden.

- **Twig and Nunjucks:** In a widely-used, open-source server-side scripting language designed primarily for web development, Hypertext Preprocessor (PHP), Twig was designed with an explicit sandbox that prohibits PHP execution to enforce the separation of concerns [29]. Similarly, Nunjucks (inspired by Python's Jinja2) provides block inheritance, autoscraping, macros, and asynchronous control, making it significantly more feature-rich than Mustache while maintaining a readable syntax [30]. It offers template inheritance with block over-riding and automatic HTML escaping for XSS prevention.
- **Jinja2:** Jinja2 is Python's fast and extensible templating engine that supports template inheritance, macros, HTML auto escaping, and sandboxed environments for safely rendering untrusted templates [3], [18]. However, the sandbox approach is fragile and prone to bypassing security. For instance, the case, CVE-2019-10906, demonstrated an incomplete blacklist; attackers bypassed the explicit block on 'str.format' by using 'str.format\_map', traversing Python's object hierarchy to reach 'built-ins' and achieve Remote Code Execution (RCE)[31]. More recently, the case, CVE-2024-56326, highlighted the difficulty of static analysis in dynamic languages, where attackers passed an indirect reference to a malicious string's format method through a custom filter [32]. Also, Parr (2004) documents that "SquareSpace" was forced to abandon an unrestricted engine after attackers exploited templates to invoke malicious class loaders [13].

### 2.2.3 Index 5:

At Index 5 with unrestricted power, these frameworks works at zero security isolation and operate at the highest level of entanglement.



## 2.2.4 Logic Friendly without Sandboxing

- **Blade and Embedded Ruby (ERB):** In the PHP system, Laravel’s native blade engine allows developers to embed arbitrary PHP expressions in the view via PHP directive, making the logic fully entangled. ERB similarly allows unrestricted Ruby execution within templates [33]. These systems offer maximum flexibility, prioritising developer speed over architectural separation.
- **Embedded JavaScript (EJS):** EJS allows developers to write JavaScript straight into templates using special ‘<% %>’ delimiters [34]. It utilises functions for fast execution, but provides no security boundaries, meaning they have direct access to JavaScript runtime, making it completely unsuitable for scenarios involving untrusted templated sources.

## Language-Integrated Template (Type-Safe Merging)

- **React/JavaScript XML (JSX) and Svelte:** JSX integrates markup directly into JavaScript a code through components [35]. Whereas Svelte shifts template processing from the browser runtime to compile-time, generating highly optimised JavaScript without framework overhead [36].

While some type-safe systems solve some data flow issues, they still struggle with preprocessing burden. In Scala/ Play, teams adopting logic-less methods found themselves creating an entirely new architectural layer, the Presenter, solely hold logic the template could no longer express [37]. This presents a problem of presentation logic does not disappear.

| System     | Computational Power | Preprocessing Burden           | Security Model       | Key Limitation        |
|------------|---------------------|--------------------------------|----------------------|-----------------------|
| Mustache   | Substitution only   | High (all logic pre-computed)  | No sandbox needed    | Bloated view models   |
| Handlebars | Limited helpers     | Medium (helpers reduce burden) | Helper whitelisting  | Helper proliferation  |
| Nunjucks   | Context-sensitive   | Low (templates handle logic)   | Sandboxing           | Separation violations |
| Jinja2     | Turing-complete     | Minimal (full Python access)   | Sandbox (bypassable) | SSTI vulnerabilities  |
| EJS        | Turing-complete     | Minimal (full JavaScript)      | None                 | Direct code injection |
| React/JSX  | Turing-complete     | Minimal (native JS)            | Type checking only   | Not logic-less        |
| Svelte     | Compile-time        | Low (compiler handles logic)   | Static analysis      | Requires build step   |

Table 2.1: Template System Comparison

Table 2.1 summarises an inverse relationship in the templating landscape: strict separation enforcement correlates with a high preprocessing burden (Mustache, Handlebars), while unrestricted computational power correlates with low preprocessing but documented security vulnerabilities (Jinja2, EJS). Overall, it can be evaluated a gap remains unsolved: no existing logic-less system provides a automated support for the preprocessing transformations it requires. This project aims to address that. In the next chapter, I will explore functional languages and how they might be used for this.

## 2.3 Functional Languages for Template Processing

Functional programming languages (Standard ML, OCaml, and Haskell) bring distinct characteristics to template processing: strong static typing catching errors at compile-time, pattern matching for data structure manipulation, and algebraic datatypes (ADTs) for representing ASTs [38], [39].

### 2.3.1 Haskell: Type-checked Metaprogramming

**Haskell**'s lazy evaluation and advanced type system have produced two distinctive templating approaches [40]. **Hamlet**, part of the Yesod web framework, uses Template Haskell metaprogramming to compile templates at build time, enabling type-checked variable substitution and ensuring that templates referencing undefined fields fail to compile rather than at runtime [41]. This shifts an entire category of template errors from production to compilation, demonstrating how sufficiently powerful type system can guarantee template accuracy.

On the other hand, **Blaze-html** represents HTML directly as composable Haskell functions rather than as a separate template language [40], [42]. Templates are not strings to be parsed but rather value constructed through variables. This eliminates the parser entirely and provides maximum type safety, but the cost of abandoning the template-as-text model that makes templates accessible to non-programmers: a trade-off that mirrors React/JSX's design choices in the JavaScript ecosystem.

### 2.3.2 OCaml: Industrial Pragmatism

**OCaml** prioritises industrial pragmatism with aggressive compiler optimisations and richer standard library via OPAM (source based package manager): combining a type system with strict evaluation and emphasis on performance [39], [43]. The Jingoo library provides Jinja2-compatible templating for OCaml applications, allowing developers to use Jinja2 syntax within OCaml programs [44]. While this offers developers familiarity from migrating from Python, it inherits Jinja2's security model, including the documented sandbox bypass concerns discussed in the earlier sections, demonstrating that functional languages hosting alone does not resolve the underlying security trade-offs of logic heavy syntax.

### 2.3.3 Elm in Elm-mustache: Closest to prior art

**Elm-mustache** is particularly relevant to this project because it is a strictly functional language with an existing Mustache implementation that explores an approach to the preprocessing problem. The library 'elm-mustache', developed by Emma Bastas and actively maintained as of 2024 [45], offers both a browser-based Mustache parser and renderer, and a command-line tool that generates type-safe Elm rendering functions from .mustache template files in build time. In its default typesafe mode, the tool produces a rendering function with an explicit context record type derived from the template's variable and section names, which the Elm compiler checks at compile time: a developer who forgets to supply a required field receives a type error before the program runs. This is the preprocessing burden made visible at the type level, exactly the guarantee that Mustache's logic-less design cannot provide when the data arrives as untyped JSON.

#### Gap

However, the typesafe mode cannot handle Mustache's list iteration. The reason is a genuine structural ambiguity: given a section with variables 'foo' and 'bar' inside it, the tool cannot determine whether those variables should be repeated for each iteration or whether they are context-wide constants, and therefore cannot produce a single correct 'Context' type without additional information [45]. To support iteration, the library offers a "full" mode that generates a runtime Mustache renderer accepts untyped Json ('Json.Decode.Value'), which supports iteration but loses the compile-time type guarantees.

This identifies the precise gap the present project fills: the typesafe approach demonstrated by 'elm-mustache' is useful for boolean sections but cannot generalise to list iteration, whereas a preprocessor that walks at typed JSON structure and augments it with contextual flags ('isFirst', 'isLast', 'hasfield') before handing it to a renderer addresses both list iteration and compile-time completeness guarantees without requiring the template to be known at build time.

### 2.3.4 Standard ML

Standard Meta Language (SML or Standard ML) provides a different outlook due to its complete specification [46]. Unlike the more expansive or experimental feature sets of OCaml or Haskell, SML's pattern matching is subject to rigorous exhaustiveness checking defined by its mathematical semantics [46], [47]. This ensures that every possible shape of data structure is accounted for during language processing. Its selection will be discussed and evaluated in design.

### 2.3.5 Takeaway

These alternatives reveal a persistent tension on the security-usability spectrum. Mustache prioritises security through logic-less design but at the cost of extensive manual data preparation [17]. Conversely, systems like Jinja2 reduce the preprocessing burden but introduce security vulnerabilities [5]. While the complaints regarding preprocessing are well-documented, no existing work formalizes these patterns into a type-safe, automated pipeline. The "iteration gap" identified in elm-mustache suggests that a more mathematically rigorous approach to data transformation is required. Addressing this, this project investigates whether the formal properties of Standard ML can automate these transformations, like specifically handling the ambiguity of list iteration as well as preserving the security advantages of logic-less templates.

## 2.4 Parsing Theory and Strategies

Parsing theory defines the process of a linear sequence of tokens and determining its grammatical structure [48], which is understandable by a computer. In a visual sense, it is the "raw parsed input" into the controller, which reads the AST, in 2.1. Within a compiler, the parser acts as a "gatekeeper" that ensures the source code follows the defined rules of the language.

This process is grounded in the *Chomsky Hierarchy*, which classifies formal grammars by their generative power and grammatical complexity into 4 levels. In 2.2, compilers are split into stages because simpler grammars are computationally "cheaper" to process. As you move from Type 3 to Type 0, the rules become less restrictive, allowing for more complex patterns but requiring more memory and processing time to decode. Type 3 in 2.2; The **Lexer** handles the simplest rules into a list of tokens using Regular Expressions (regex) to identify the vocabulary of the code without memory of the previous steps beyond its current state [49]. Type 2 in 2.2; The **Parser** takes tokens from Type 3 and organises them into an AST by using a stack based pushdown automaton, which provides the recursive memory required to validate nested structural dependencies [49]. Type 1 in 2.2; The **Semantic Analyser** enforces context sensitive rules that a tree structure alone cannot verify, ensuring logical consistency. Type 0: **Unrestricted Grammars** represent the absolute limit of computational power to a Turing machine, defining the theoretical boundaries and logic of the final compiled executable, thus it is outside the compiler.

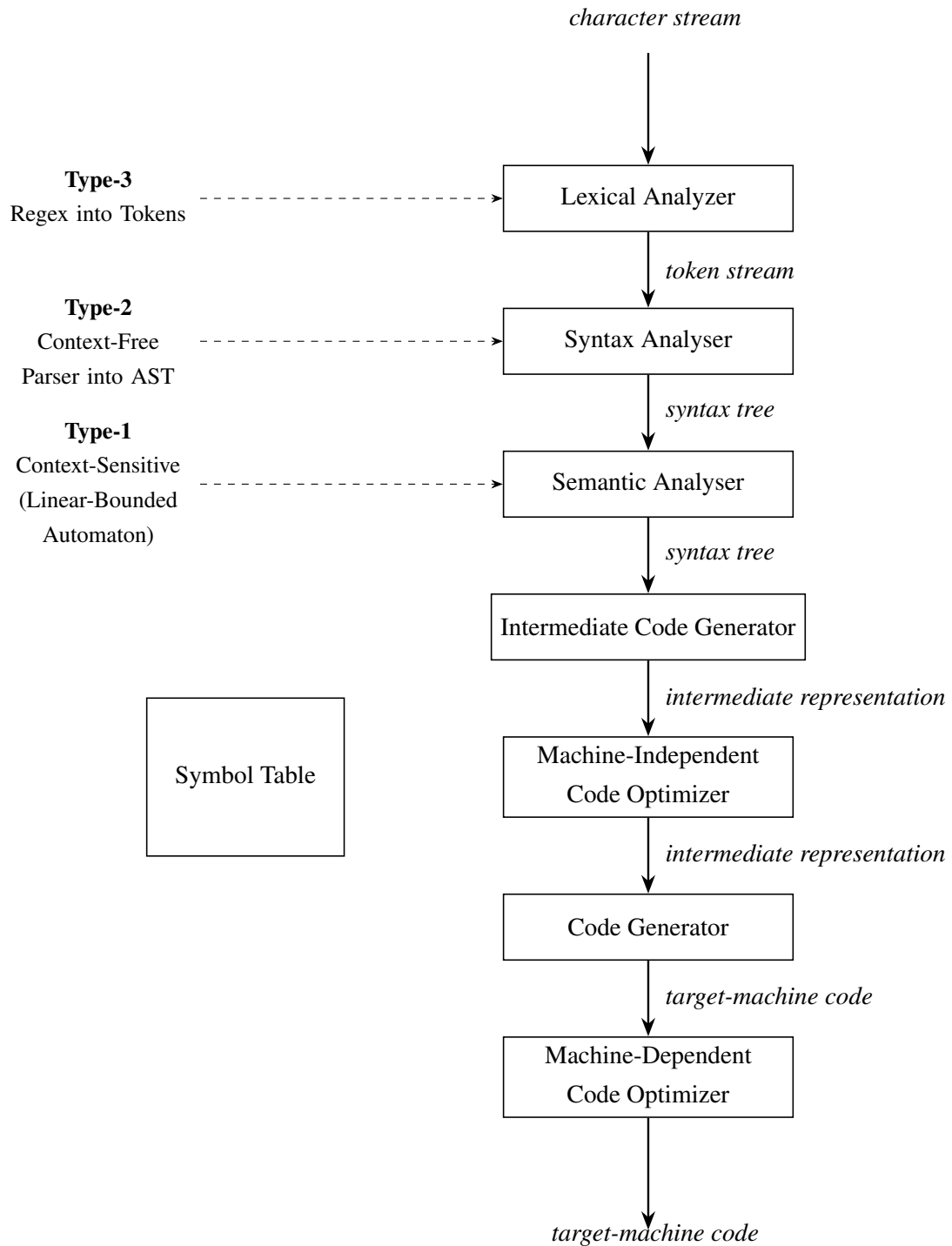


Figure 2.2: Mapping of compiler phases to the "Chomsky hierarchy". The diagram illustrates the increasing levels of computational complexity required for early-stage analysis: from Regex pattern-matching tokens (Type-3) to validating hierarchical structure (Type-2) and enforcing contextual logic (Type-1). In reality, some phases may be combined. Adapted from [48].

When you provide a mustache template, which looks like

```
"{{#section}}{{name}}{{/section}}",
```

the computer recognises it as ASCII characters of which the parsing theory breaks this down.

- **Lexer:** This scans the group of characters into "tokens" ie `OPEN_SECTION`, `NAME`, `CLOSE_SECTION`.
- **Parser:** This arranges those tokens into an AST, which is a logical representation of the structure.

There are many parsing strategies but the primary ones are "Top-Down (LL)", which expand rules from the start symbol or "Bottom-Up (LR)", which utilise a "shift reduce" mechanism to build the tree from the input tokens [48]. This project employs LALR (Look-Ahead LR) algorithm, which will be discussed in Chapter 4 (Design).

The parser uses lookahead ( $k$ ) to resolve grammar conflicts by inspecting upcoming tokens before committing to a rule. Specifically, LALR(1) provides a layer of "syntactic safety" by blocking invalid inputs from reaching the execution stage [50].

Ultimately, parsing is foundational to computer science because it bridges the gap between human-readable text and machine-actionable logic. Without a parser, a program is just a meaningless linear sequence of characters. By enforcing strict context-free grammar rules, a parser actively resolves structural ambiguity, providing mathematical certainty that the code is safe to execute and structural understanding. In the specific context of this project, defining a custom parser allows the system to overcome the "iteration gap" identified in previous implementations like `elm-mustache`, granting complete structural control over how the template is interpreted and rendered.

## 2.5 Parser Generators: ML-Lex and ML-Yacc

Having established the theoretical mechanisms of how parsing operates, the next architectural consideration is its implementation. While the mathematics of LALR state machines and lookahead are rigorous, manually writing out thousands of shift-reduce states is highly inefficient and susceptible to human error. To bridge this gap between formal theory and practical software engineering, developers automate the process using **parser generators**.

To summarise some selected parsing generators, Table 2.2 provides a comparative overview of some relevant generators. Specifically, it highlights:

- **Yacc and Bison)** One of the earliest parser generators and its modern GNU successor, serve standards for LALR(1) parsing in C/C++ environments.
- **Menhir / ML-Yacc:** The foundational LR(1) and LALR(1) tools tailored for functional programming ecosystems.
- **ANTLR:** A versatile, multi-language powerhouse utilizing Adaptive LL(\*).

- **Tree-sitter:** A modern, incremental GLR parser designed specifically for real-time IDE integration.
- **ML-LPT (ML-ULex and ML-Antlr):** The modern, Unicode-aware successor to ML-Lex and ML-Yacc for Standard ML [51]. It utilises top-down  $LL(k)$  parsing to produce purely functional code that is intuitive to debug, though it requires refactoring left-recursive rules.
- **Parser Combinators:** A highly flexible, monadic approach that embeds the grammar directly within host-language code (e.g., Haskell) as higher-order functions [52]. While eliminating the need for separate specification files, they typically lack static conflict detection and are generally weaker than LALR(1) [53].
- **CM-Lex and CM-Yacc:** SML/NJ tools tightly integrated with the Compilation Manager (CM) to mitigate strict compilation order errors [54]. They generate closed functors, catch ambiguities at generation time, and automatically track dependencies to prevent manual recompilation [55].

By contrasting their underlying parsing algorithms, supported output languages, and primary use cases, the table illustrates the broad spectrum of tools currently available to developers, ranging from strict, low-level tokenisation to highly fault-tolerant, multi-language syntax analysis.

| Tool               | Algorithm                    | Target Language                                        | Output & Integration                                               | Key Limitation                                             |
|--------------------|------------------------------|--------------------------------------------------------|--------------------------------------------------------------------|------------------------------------------------------------|
| Yacc               | LALR(1)                      | C                                                      | Grammar/code mixed; external lexer (lex)                           | LALR(1) only; C output only                                |
| Bison              | LALR(1), LR(1), IELR(1), GLR | C, C++, D, Java                                        | Grammar/code mixed; external lexer                                 | C-family targets only                                      |
| ANTLR4             | Adaptive LL(*)               | C#, Java, Python, JavaScript, C++, Swift, Go, PHP      | Grammar/code separate; lexer generated                             | JVM-based toolchain                                        |
| Menhir             | LR(1)                        | OCaml                                                  | Grammar/code mixed; lexer generated                                | OCaml output only                                          |
| Tree-sitter        | LR(1), GLR                   | C, with bindings (Rust, Wasm, JavaScript, Python, ...) | Grammar/code separate; lexer generated + external                  | Native output C; other languages via bindings only         |
| JavaCC             | LL( $k$ )                    | Java, C++, JavaScript (via GWT)                        | Grammar/code mixed; lexer generated                                | JVM-based toolchain                                        |
| Happy              | LALR, GLR                    | Haskell                                                | Grammar/code mixed; external lexer                                 | Haskell output only                                        |
| ML-Antlr (ML-LPT)  | LL( $k$ )                    | Standard ML                                            | Purely functional output; Unicode-aware lexer (ML-ULex)            | LL grammars cannot directly express left-recursion         |
| Parser Combinators | Higher-order combinators     | Host language (e.g. Haskell)                           | Grammar embedded in host code; no separate spec file               | No static conflict detection; weaker than LALR(1) for CFGs |
| CM-Lex / CM-Yacc   | LR-based <sup>†</sup>        | Standard ML (SML/NJ)                                   | Generates closed functors; integrates with the Compilation Manager | Tied to the SML/NJ CM ecosystem                            |

Table 2.2: Comparison of parser generators, taken from [56].

While formal theoretical models rely on automated tools to generate finite-state and push-down automata, many production-grade compilers abandon generated code in favor of hand-



written implementations [57]. Most notably, syntax analysis (Type-2) is frequently executed via a recursive descent parser [48]. Rather than maintaining an explicit, isolated stack data structure to process context-free grammars, a recursive descent parser leverages the underlying execution call stack of the host programming language through mutually recursive functions [48]. This manual approach provides developers with granular control over the parsing state, allowing for highly optimised performance, superior error recovery, and the seamless integration of context-sensitive (Type-1) semantic checks directly into the syntax analysis phase [58].

### 3. Requirements and Analysis

#### 3.1 Problem Statement

| (a) A section of Input:<br><i>data.json</i>                                                                                                                                                          | (b) Template:<br><i>example.mustache</i>                                                                                     | (c) Rendered output                                                                                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <pre>{   "name": "JohnSmith",   "email":     "john@example.com",   "degree":     "BSc Computer Science",   "skills": [     {"skill": "Python"},     {"skill": "SQL",       "last": true}   ] }</pre> | <pre># {{name}}  Email: {{email}}  Degree: {{degree}}  Skills: {{#skills}} - {{skill}} {{/skills}}  Summary: "example"</pre> | <pre># JohnSmith  Email:   john@example.com  Degree:   BSc Computer Science  Skills: - Python - SQL  Summary: example</pre> |

*Figure 3.1: The visual workflow before this project. A raw dataset (a) is hand-organised into JSON and passed through a logic-less Mustache template (b) to produce a simple README (c)*

While "logic-less" templating engines like Mustache can mitigate SSTI by separating business logic from presentation view; they introduce a significant preprocessing burden. This project addresses this burden when the developer must manually shape data to fit restricted template.

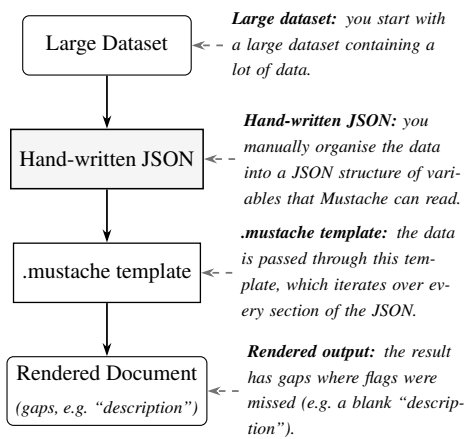
An example of the workflow is illustrated in 3.1. As shown in the input `data.json` (Figure 3.1a), the developer must manually inject contextual flags, such as `"last": true`, to handle formatting logic that the engine cannot compute itself. Furthermore, if a template contains static placeholders or missing fields, such as the `Summary: "example"` tag in Figure 3.1b, the developer is forced into a repetitive cycle of manual intervention, they must either manually edit the rendered output after generation, return to the source JSON to add missing data or leave it blank. This "recursive fixing" becomes increasingly labor-intensive as dataset scales or number of files increases.

3.2 illustrates an comparison of the manual route with the proposed automated pipeline. The current manual process (Figure 3.2, left) relies on "Hand-written JSON", a step that is tiresome, error-prone and lacks any formal verification. This leads to three primary shortcomings:

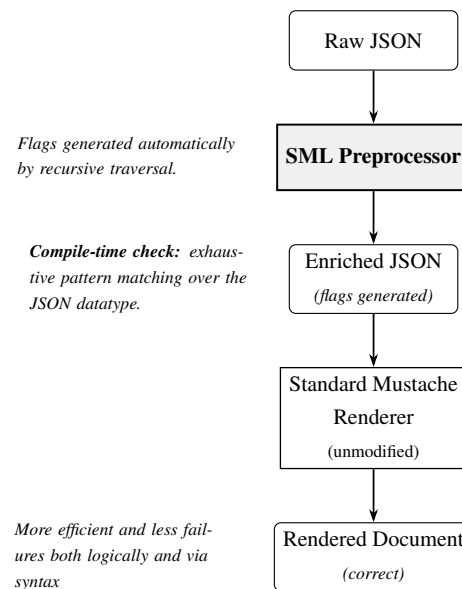
- Not clear errors: Highlighted in 3.2, errors in manual preprocessing, like missing variables, does not trigger compile-time or runtime warnings

- **Manual Maintenance:** Every structural change to `.mustache` may require a corresponding manual update to the JSON, and vice versa. In a manual system, the developer must identify every new contextual requirement by hand, which can lead to significant toil as the application evolves. This signals a potential for efficiency.
- **Lack of Verification:** There is currently no mechanism to ensure the input data satisfies the structural requirements of the template. While the parsing phases of a compiler rely on the Chomsky Hierarchy (Type 2) to ensure grammatical correctness 2.2, logic-less templates lack a similar validation method to verify the data context is complete before rendering begins.

#### Without this project (manual)



#### With this project (automated)



**Figure 3.2: Manual preprocessing (left) versus the proposed automated preprocessor (right).** The manual route introduces maintenance overhead, no verification mechanism, and silent failures detectable only by inspection; the preprocessor removes the hand-written step and converts an entire class of omission error into a compile-time error, while the Mustache renderer remains unmodified in both cases.

This project addresses these shortcomings by developing an automated preprocessor, 3.2. The tool automatically generates the necessary contextual flags to produce an "enriched JSON" document, using a raw JSON document to better fit the mustache template. By sitting "in front" of the renderer, this prototype preserves the security benefits of mustache specification while utilising a pattern-matching language to ensure necessary variables is applied consistently.

### 3.2 The Task:

Here, the task is specified into further technical requirements, breaking down into manageable components, analysing the chosen approaches and establishing the requirement criteria by which the final implementation will be evaluated.

1. **Templating Parsing:** The system must be able to parse standard Mustache templates, converting the desired strings into an AST, which can be used as an indication of what is desired. This must accurately reflect the JSON value space (objects, arrays, strings, numbers, booleans, and nulls).
2. **Reading JSON data:** The system must read raw JSON input (via file or string) and map it to an internal algebraic datatype. This must accurately reflect the JSON value space (objects, arrays, strings, numbers, booleans, and nulls).
3. **Contextual Enrichment:** The system must automatically transform the ingested JSON by applying contextual logic rules: for example, null fields may be replaced with an empty string if there is no information.
4. **Template Rendering:** The system must render the parsed AST against the enriched JSON context to produce a final output string. The renderer must resolve dot-separated paths, process implicit iterators, evaluate truthiness for section blocks, and handle HTML escaping where required similar to mustache syntax.
5. **Command-Line Interface:** The system must provide a unified CLI, allowing a user to provide a template file and a JSON data file as arguments, producing the rendered string to standard output.

For our case, I have chosen the implementation output language to be Python, as mustache has more than 40+ implementation languages [59]. The reason why Python was chosen as the target language is due to wide-scale and rising use. Python is especially well-suited for machine learning and artificial intelligence and is particularly approachable due to its easy syntax [60].

### 3.3 Requirements

To ensure the system is robust, maintainable and verifiable, the non-functional constraints must be met:

1. **Compile time Correctness:** The preprocessing transformations must be implemented, ie using exhaustive pattern matching.
2. **Data Preservation:** Enrichment must be solely additive. Existing data types, values, and ordering must remain unaltered, with the exception of something being empty.

3. Deterministic: identical inputs must universally produce identical rendered outputs without reliance on hashing or arbitrary map iteration orders.
4. Separation: The architecture must strictly separate the Parser, Preprocessor, and Renderer components to allow for independent testing and future extensibility.

### 3.4 Breakdown

This is a breakdown of implementation. Design considerations will be discussed more in the next chapter.

1. Build a parser: this will decipher the mustache template into a structured AST using lexers and parsers.
2. Enrich the JSON: Automatically inject presentation syntax (e.g., positional flags and null-handlers) into the data model.
3. Build a renderer: this will combine the parsed AST with the enriched JSON context to generate the final output in python.

### 3.5 Testing and Evaluation

The success of this project depends on two distinct factors: the technical correctness of the preprocessor i.e. whether has an impact on reduction of preprocessing burden and its practical utility in a real-world workflow, i.e. how easy is it to implement. It should measurably improve the developer experience by reducing the preprocessing burden identified in the problem statement.

The following sections outline how the system was verified and the criteria used to evaluate its impact

#### Component-Level Testing

Because the system is split into separate modules, I tested each one on its own while I was writing it, using the SML/NJ REPL. This kept the feedback loop short: a change to the parser could be checked straight away without running the renderer. The tests fall into three categories for every stage of the pipeline

**Parser Testing** These validate that the parser builds the correct AST across a range of inputs, with particular attention to deeply nested sections and to Mustache's standalone-line newline-stripping rules.

**Preprocessor Testing** These verify that enrichment recurses properly into nested data, and that the position flags (`isFirst`, `isLast`) and the `has_x` booleans come out right

at the boundaries: empty lists, single-element lists, and the first and last elements of a list but also could add external logic, like "description".

**Renderer** These cover dotted path lookups, the truthiness rules that decide whether a section is rendered, and HTML escaping and follows up on Mustache syntax

Ensuring the first round of tests were unit validates that each stage is working properly.

Following these unit tests, it is important to include automated implementation tests, with real and different examples.

## Integration and Automated Implementation Testing

- **Does this follow mustache syntax?** The system was evaluated against the official Mustache specification [59]. This involved executing a series of test cases defined in JSON, ensuring that the SML/NJ implementation adheres to the cross-language standards for whitespace handling, partials, and complex scoping.
- **Does this add value in terms of logic?** These tests verified that the “contextual enrichment” performed by the Preprocessor (e.g., null-to-string transformation) correctly informed the Renderer’s logic. This confirmed that the internal data transformations remained consistent as they passed through the architectural boundaries of the system.
- **Is this able to handle scale?** Automated scripts were used to inject large-scale JSON datasets into the system. Handling scale is important because templates are used everywhere even in large scale SDK generation. This served to validate the efficiency and versatility of the project on whether it is able to adjust to large datasets.

## Evaluation Criteria

I assessed the success of the system based on three main objectives:

1. **Eliminating Manual Work** Does the preprocessor actually stop the need to manually fix JSON lists and null values, or is the effort just being shifted elsewhere?
2. **Compatibility** Can the processed JSON still be used by a standard, unmodified Mustache renderer? The goal was to add functionality without breaking existing tools.
3. **Reliability** Does SML’s exhaustive pattern matching actually catch the edge cases that caused silent failures in the manual workflow? I wanted to see if the compiler could provide a safety net that was missing before.

### 3.6 Legal and Ethical Considerations

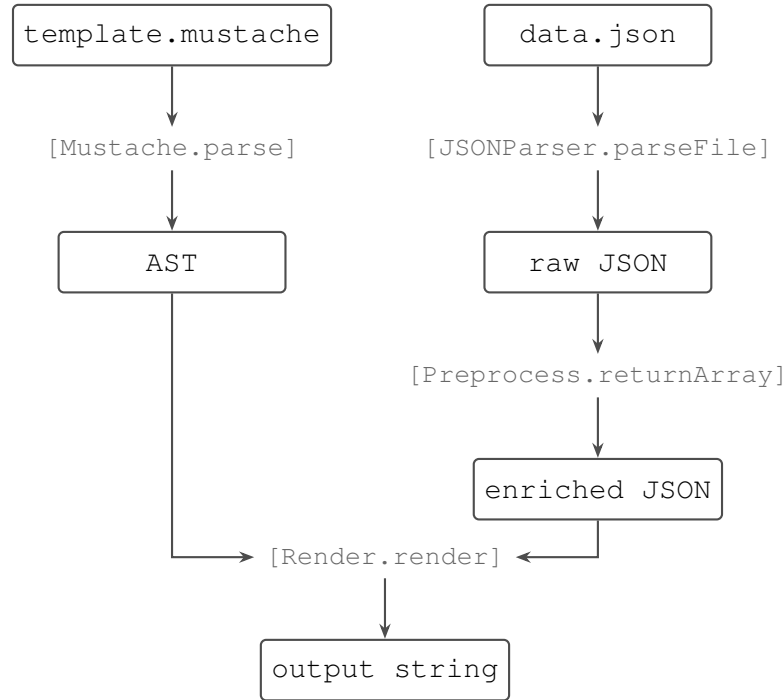
This project builds only on open-source libraries, the SML/NJ standard library and its JSON parser, all distributed under permissive licences that allow use and redistribution for academic work. The tool runs locally and only ever touches static templates and structured data supplied by the developer; it does not collect, transmit, or store any personal data, so there are no data-protection issues to manage.

There is also a modest security upside. By keeping the logic-less model intact and moving the data shaping into a separate, type-checked preprocessing step instead of into the template itself, the tool stays clear of the patterns that lead to SSTI. While this preprocessor is not a security tool, this project aims to minimise the risk of vulnerabilities like SSTI or RCE by keeping the transformation logic strictly data-focused. It does not evaluate arbitrary code or execute external scripts.

However, from an ethical standpoint, it is important to note that this system has only been validated against a small, controlled repository. It has not undergone a formal security audit. Therefore, using this tool in a production environment with untrusted or "dirty" input carries an inherent risk, and I do not recommend its use in security-critical systems without further testing.

## 4. Design

Chapter 3 (requirements) identified the existing problem space and evaluated approaches for automating Mustache. In this chapter, those approaches mentioned are justified with reasons in further detail. The implementation follows a pipeline architecture, which highlights the steps from template parsing to JSON enrichment and final rendering 4.1.



*Figure 4.1: The preprocessing pipeline. The Mustache template is parsed into an abstract syntax tree (AST); the JSON data is parsed into a raw JSON value; the preprocessor walks the raw JSON and emits an enriched JSON value containing the contextual flags the template requires; the renderer consumes both the AST and the enriched JSON to produce the final output string.*

### 4.1 Choice of Implementation Language

The literature review highlighted how functional languages has specific properties that would be good to exploit, like how it can catch errors in the compiler and, in terms of scalability, utilise immutability to enable parallel processing and a type system to eliminate entire classes of runtime bugs.

After an investigation into Standard ML, OCaml, Haskell, and Elm as potential functional languages to utilise for this project, SML was chosen for 3 primary reasons:

1. Formal specification: Standard ML has a complete mathematical definition of its typing rules and operational semantics [46], and multiple standard-conforming implemen-



tations exist (SML/NJ, MLton, MLKit, Poly/ML) [61]. For a project whose central desires type-checked correctness, working in a language with a formal specification means that the guarantees the type system provides are precisely defined, not just behaviour observed in one implementation.

2. Pattern Matching: Having the compiler enforce checks on all possible data variants is useful for mustache tags and to ensure every JSON value is handled.
3. The system: The use of signatures, functors and lazy streams allows for optimal building of a modular system that can process massive amounts of data without hogging. This will be discussed further later this chapter on the details that make this optimal.

A disadvantage of choosing Standard ML is that its ecosystem offers far fewer third-party libraries than Python or JavaScript, and noticeably fewer than OCaml or Haskell. This project depends on the SML/NJ JSON library, which proved sufficient for the preprocessing task but has a much smaller user base than equivalents such as Python's 'simplejson' or Haskell's 'aeson'. As a result, fewer edge cases have been encountered and resolved by the wider community, and the available documentation is comparatively sparse

A dynamically-typed language was rejected for the obvious reason: the project's contribution is precisely the use of a static type system to catch preprocessing errors at compile time. A Python preprocessor could do all the same data transformations, but it would not provide the exhaustiveness guarantees, which is the whole point.

Elm, which already has a Mustache implementation [45], was the most serious alternative. Elm pairs a strict type system with logic-less templates and demonstrates that the architectural pairing is viable. However, Elm is primarily a frontend language compiled to JavaScript, and its Mustache library does not automate preprocessing transformations: developers still write the data construction by hand. Standard ML, potentially used as a backend preprocessing tool, lets the project focus on automating the transformation rather than on browser integration.

## 4.2 Pipeline Choice

Three architectural patterns have been evaluated: modifying an existing Mustache engine, building a new engine from scratch, or building a preprocessor to sit in front of a standard engine.

The preprocessor-led pipeline was chosen. Building a new engine alone would not solve the underlying issue of "fragile" JSON data, and modifying an existing C or JavaScript engine would sacrifice the type-safety benefits of SML. The preprocessor approach allows the tool to enrich the data before it reaches the renderer.

The second major decision was where to put the tool in the rendering pipeline. There were three options:

1. Modify an existing Mustache implementation to add the missing logic.
2. Build a new template engine that supports the missing operations directly.
3. Build a preprocessor that sits in front of any unmodified Mustache implementation.

I chose the third. Each of the alternatives had a problem that ruled it out for this project after experimentation.

Modifying an existing Mustache implementation cause problems and lose mustache's advantages. Of which, Mustache is interesting precisely because the same template can be rendered by implementations in dozens of languages. This would not work with its syntax of being 'logic-less', alternatively would mean just manual labour to build a new templating engine.

Building a new template engine would make it possible to support conditionals, arithmetic, and other operations directly in the template, but doing so would re-introduce exactly the security risks identified [5], [13]. The motivation for working with Mustache is its logic-less design; an engine that adds logic would not preserve that property and would defeat the purpose. Additionally Mustache's versatility into many languages was one of its strong suits, it made more sense to utilise this for the long-run.

The JSON data would be operated on before any template engine sees it, as how mustache is operated now where JSON comes before mustache and the engine itself remains unchanged, retraining its advantages. An altered output is just JSON, with the contextual flags embedded as ordinary fields, so it works with the JavaScript reference implementation, the Ruby implementation, the Haskell implementation, or any other. The Mustache specification is preserved.

The trade-off is that the preprocessor cannot help with logic that Mustache itself rejects or anything that is not specified in the Mustache template by the developer. This ensures that what is rendered is what is desired by the developer. Thus, what is needed and wanted must be conveyed by the developer in the mustache template; this reduces any unnecessary details that is unwanted.

### **4.3 Parser Design**

The parser is the most technically complex component, as it must transform raw text into a structured AST.

### 4.3.1 Choice of Parser Generator

I used `CM-Lex` and `CM-Yacc` rather than hand-writing a recursive-descent parser or using combinators. As noted by Crary [55], these tools allow for an explicit grammar definition in separate specification files, which provides generation-time conflict detection. This ensures the grammar is unambiguous before a single line of SML is compiled and does not interfere with the code if it is incorrect.

### 4.3.2 SML Mechanisms and Functors

Unlike traditional tools like `Lex` and `Yacc`, which often suffer from the "disembodied code problem", `CM-Lex` and `CM-Yacc` generate closed functors [55].

In this implementation, the lexer and parser are defined as functors that are instantiated with specific action code. This allows for:

- **Type-Safe Actions:** Every snippet of code executed when a token is found is type-checked within its proper context.
- **Lazy Streams:** The lexer utilise lazy functional streams (`Stream.stream`), allowing the system to process large templates efficiently without loading the entire file into memory at once [55].

### 4.3.3 AST Shape

The AST is defined in `'ast.sml'` as a single algebraic datatype with nine constructors:

```
structure Ast =  
struct  
  datatype node  
    = Text of string  
    | Variable of string  
    | UnescapedVariable of string  
    | Section of string * node list  
    | InvertedSection of string * node list  
    | Comment of string  
    | Partial of string  
    | DelimiterChange of string * string  
    | Template of node list  
end
```

This shape was chosen because it maps directly onto Mustache's surface syntax. Each constructor corresponds to one tag type in the specification, which makes the parser's job mostly mechanical: each grammar production emits exactly one node constructor. The only constructors that hold recursive structure are `'Section'`, `'InvertedSection'`, and `'Template'`, which

is the minimum needed to express arbitrary nesting.

Two of the constructors ('Partial' and 'DelimiterChange') are present because the parser recognises their syntax, but the renderer does not process them. These were chosen to keep them in the AST rather than reject them at parse time so that the parser remains a faithful representation of Mustache syntax, with potential adjustments made later. This means the AST can be reused by future work that does implement partials or delimiter changes, without re-parsing.

## 4.4 Preprocessor Design

The preprocessor is the core contribution of this project. Rather than operating on strings, it operates directly on an SML representation of JSON.

The logic follows a recursive traversal of the JSON tree. When it encounters a list or a null value that would traditionally break a Mustache template, it applies a transformation to "wrap" or "enrich" the data. For example, a null value is converted into a boolean-false or an empty list, preventing the renderer from failing silently. This approach leverages SML's type system to guarantee that no branch of the JSON tree is left untouched.

The preprocessor operates on the SML/NJ JSON library's 'JSON.t' datatype rather than on JSON as a raw string. This is important: pattern matching on the datatype is what makes the type-system guarantees work. A string-based preprocessor could perform the same transformations using regular expressions, but the compiler would have no way to verify that every JSON value type was handled. By operating on the typed datatype, the compiler verifies exhaustiveness over 'OBJECT', 'ARRAY', 'STRING', 'NUMBER', 'BOOL', and 'NULL'. If the JSON library adds a new constructor in a future version, the preprocessor will fail to compile until that case is handled. This is precisely the property the project is designed to demonstrate.

The preprocessor exposes a small set of functions:

```
val wrapItem      : JSON.t -> bool -> bool -> JSON.t
val flagList      : JSON.t list -> JSON.t list
val processFields : (string * JSON.t) list -> (string * JSON.t) list
val processValue  : JSON.t -> JSON.t
val returnArray   : JSON.t -> JSON.t
```

'wrapItem' attaches 'isFirst' and 'isLast' flags to a single value. 'flagList' applies it across a list, computing the correct flags for each position. 'processFields' handles object fields, applying the null-rewriting rule. 'processValue' is the recursive entry point. 'returnArray' is the public-facing function that consumers call.

'processFields' and 'processValue' are mutually recursive, which is what allows the transfor-

mation to descend through arbitrarily nested JSON in a single pass. Each function handles one level of structure (object fields, or any value), and they call each other to walk into nested contents.

#### 4.4.1 A Worked Example

The following example illustrates all the transformation rules acting together. The input contains a present field, a null field, and an array of strings:

```
{
  "name": "Snow Leopard",
  "description": null,
  "threats": ["habitat loss", "poaching"]
}
```

After 'Preprocess.returnArray', the same data has been enriched with contextual metadata:

```
{
  "name": "Snow Leopard",
  "has_name": true,
  "description": "",
  "has_description": false,
  "threats": [
    { "value": "habitat loss", "isFirst": true, "isLast": false },
    { "value": "poaching", "isFirst": false, "isLast": true }
  ],
  "has_threats": true
}
```

A Mustache template can now use '{{#has\_description}}' to render an optional block, or '{{^isLast}}, {{/isLast}}' to suppress the trailing comma on the last array element. None of this logic exists in the template, and none of it had to be computed by the application code.

## 4.5 Renderer Design

The renderer is a tree-walking interpreter. It takes the AST generated in Section 4.3 and the enriched JSON from Section 4.4 to produce the final string output. I implemented a custom "truthiness" definition to ensure that the renderer handles SML-transformed JSON values in a way that matches user expectations (e.g., empty lists and nulls evaluating to false).

The renderer is structured around a single recursive function that dispatches on each node type. Each constructor has exactly one branch, and the SML compiler verifies that no branch is missing. This direct mapping from AST shape to renderer behaviour makes the renderer's correctness easy to argue.

### 4.5.1 Truthiness

Mustache’s specification leaves some latitude in what counts as a “falsy” value. I chose the following definition, taken verbatim from `renderer.sml`:

```
fun isTruth JSON.NULL          = false
  | isTruth (JSON.BOOL false)  = false
  | isTruth (JSON.ARRAY [])    = false
  | isTruth _                  = true
```

A value is falsy if it is JSON null, the boolean `false`, or the empty array; everything else is truthy. This matches the behaviour of most mainstream Mustache implementations.

The empty-array case is the most subtle of the three. Without it, `'{{#items}}...{{/items}}'` would render its body once for an empty array, which is almost never what a template author wants. Including the empty-array case in the falsy set means that `'{{^items}}No items.{{/items}}'` works correctly as a fallback message.

### 4.5.2 Dot-Separated Path Resolution

The renderer supports dot-separated paths such as `'{{person.name}}'` in addition to single-segment lookups. Real JSON data is typically nested, and forcing template authors to flatten their data structure before rendering would re-introduce a form of preprocessing burden, which is the opposite of what the project is trying to do. Supporting dotted paths costs only a handful of lines in `resolve` and `lookupKey` and broadens the range of templates the system can render usefully.

## 4.6 Build and Deployment

The final design decision was how to package the tool: as a library callable from other SML code, or as a command-line tool. I chose the command-line tool because the most common use case for a preprocessor of this kind is shell-based scripting:

```
./render template.mustache data.json > output.html
```

This is the same workflow used by other Mustache command-line tools and by classic Unix text processors, so it fits naturally into existing toolchains. Wrapping the pipeline in a single binary also means users do not need to know any Standard ML to use the tool.

The build process uses `ml-build` to produce a heap image, which is then launched by a short shell wrapper. The heap image’s filename suffix is platform-specific (`.amd64-darwin` on macOS, `.amd64-linux` on Linux), which the wrapper currently hard-codes. This is a known limitation, documented in Section 3.5, and would be straightforward to fix using the `'$SMLNJ_ARCH'` environment variable in future work. I did not fix it during the project

because it is a deployment friction rather than a correctness issue, and the time was better spent elsewhere.

## 5. Implementation and Testing

This chapter describes how the design from Chapter 4 was actually built, the implementation challenges that took meaningful time to resolve, and the testing carried out at each stage. The full source code is available in GitHub [62].

### 5.1 Implementation Overview

The project is implemented in Standard ML and uses three external tools: SML/NJ (Standard Meta Language/ New Jersey) as the compiler, CM-Lex and CM-Yacc as the lexer and parser generators, and Git for version control. There are 3 directories, which link to each stage of the proposed pipeline 4.1: 'mustache\_parser' for the lexer, parser, AST, and orchestration code; 'preprocessing' for the JSON enrichment; and 'renderer' for context resolution, truthiness, and the final script. Each directory contains its own test, runnable independently from the SML/NJ REPL, and readme instructions.

Development was carried out on macOS using SML/NJ version 110.99.9. One note of the local environment is that the system's default 'sml' command invokes Poly/ML rather than SML/NJ, so SML/NJ has to be invoked through its absolute path. This matters because the SML/NJ JSON library used by the preprocessor is not available under Poly/ML. The build script handles this internally, but it is documented here because anyone reproducing the build will hit the same issue.

### 5.2 Implementing the Parser

The parser was the first component built. Every other component depends on the AST it produces, and my supervisor had recommended using CM-Lex and CM-Yacc as a learning vehicle for parser generators before doing anything else.

The lexer, defined in 'mustache.cmlex', scans the input character stream and emits tokens for each Mustache construct. It is implemented as a lazy stream, which means tokens are only produced when the parser asks for them. Lazy stream is part of the SML manual. This was conceptually unfamiliar at first, because lazy evaluation in SML requires a specific construction: a thunk of type 'unit -> 'a front' wrapped inside a 'Stream' constructor. The 'Stream' wrapper is not a design choice but a syntactic requirement of SML. Mutual recursion needs the wrapper to type-check correctly. Once the pattern was understood it became straightforward to extend.

The parser, defined in 'mustache.cmyacc', consumes the token stream through a context-free



grammar with two nonterminals ('Template' and 'Node') and ten productions. Semantic actions in the grammar invoke constructor functions ('mk\_text', 'mk\_section', 'mk\_inverted', 'mk\_variable', and so on) that build AST nodes bottom-up as the parser reduces. One non-obvious detail of CM-Yacc is that grammar precedence numbers behave the opposite way to the BIDMAS (order of brackets, indices, division, multiplication, addition, subtraction) convention.

'Parser' wires the lexer and parser together and exposes the public interface 'Mustache.parse'. It also implements two helpers: 'extractName', which strips whitespace, and 'stripLeadingNewline', which removes the leading newline from the body of a section so that templates do not produce blank lines around section boundaries.

### 5.3 Implementing the Preprocessor

The preprocessor lives in 'preprocessing/preprocess.sml' and is the core contribution of the project. It consists of three main functions: 'processFields', 'flagList' (helped by 'wrapItem'), and 'processValue'.

'processFields' walks the fields of a JSON object. For each field, if the value is 'JSON.NULL', the value is replaced with an empty string and a companion field of the form 'has\_x = false' is added (where 'x' is the original field name). If the value is anything else, it is kept and the companion field is set to 'has\_x = true'. This makes optional fields expressible in a Mustache template as '{ {#has\_name}} ... {/has\_name} '.

'flagList' walks a JSON array and rewraps every element so that it carries 'isFirst' and 'isLast' boolean flags. The wrapping is done by 'wrapItem', which takes the original value and the two flags and produces a new 'JSON.OBJECT' containing them. The implementation tracks position by pattern-matching on the head/tail structure of the list: the first element gets 'isFirst = true', the last element gets 'isLast = true', and everything else gets both flags set to false.

'processValue' ties the two transformations together by recursively walking the entire JSON tree. Whenever it encounters an 'OBJECT', it applies 'processFields' and then recurses into the object's values. When it encounters an 'ARRAY', it applies 'flagList' and then recurses into each element. This recursive traversal is the reason deeply nested structures get enriched correctly, and it is also the part of the code where the exhaustiveness check earns its keep, every branch over the 'JSON.t' datatype must be present, which makes it impossible to forget a case.

The public interface is 'Preprocess.returnArray', which takes a 'JSON.t' and returns a 'JSON.t' with all transformations applied. Exposing only this single function keeps the rest of the codebase decoupled from the internal helpers.

## 5.4 Implementing the Renderer

The renderer, in `'renderer/renderer.sml'`, takes a parsed AST and an enriched JSON value and produces the rendered output string. It consists of four pieces: a context resolver, a truthiness function, an HTML escape function, and the main `'renderNode'` walker.

`'resolve'` and `'lookupKey'` together implement context lookup. They support dot-separated paths (so `'person.name'` walks into the `'person'` object and then reads its `'name'` field) and the implicit iterator `'.'` (which refers to the current item when iterating over an array).

`'isTruth'` decides whether a value triggers a section block. The rule is given in Section 4.5.2.

`'htmlEscape'` replaces the five HTML-significant characters (`&`, `<`, `>`, `"`, `'`) with their corresponding entity references. It is called only on the output of `'Variable'` nodes, not `'UnescapedVariable'` nodes, which is why Mustache distinguishes the two with double versus triple braces.

`'renderNode'` is the main walker. It takes a node and a context and produces a string. `'Text'` nodes return their stored text directly; `'Variable'` nodes look up the name and HTML-escape it; `'UnescapedVariable'` nodes do the same without escaping; `'Section'` nodes either iterate over an array (rendering the body once per element with the element as the new context), render the body once for a truthy non-array value, or skip entirely for a falsy value; `'InvertedSection'` nodes render only when the value is falsy; `'Comment'` nodes return the empty string, and `'Template'` nodes concatenate the output of their children.

The public interface is `'Render.render'`, which takes a template string and a `'JSON.t'` and returns a string. Internally, it parses the template, runs the preprocessor on the JSON, and walks the resulting AST.

## 5.5 Coding Issues Encountered

Several issues took time to resolve and are documented here.

**CMtool installation** The CMtool source distribution did not include the `'Makefile'` and `'calc.cm'` files referenced in the documentation, and the generated lexer and parser file names (`'calc.cmlex.sml'`, `'calc.cmyacc.sml'`) differ from the names suggested in older tutorials (`'calc.lex.sml'`, `'calc.parse.sml'`). Resolving this required reading the generator output rather than trusting the documentation, and writing the build configuration files by hand.

**Unbound signatures with the standard library** On a clean install, `'$/smlnj-lib.cm'` did not resolve correctly when used inside `'calc.cm'`, producing an unbound signature error. The fix was to hard-code the absolute path to `'cmlib.cm'` inside the project directory. This is a portability issue rather than a logic bug, but it consumed non-trivial time.

**Default 'sml invokes the wrong implementation.** As already noted, the default 'sml' command on this development machine invokes Poly/ML, not SML/NJ. Since the JSON library is specific to SML/NJ, the build and REPL workflows always require the absolute path '/usr/local/smlnj/bin/sml'. A 'loadrender.sml' helper file was written to automate REPL setup, but its paths are hard-coded and do not work on non-standard installs.

**Platform-specific heap image suffix.** The shell wrapper that launches the renderer hard-codes '.amd64-darwin' as the heap image suffix. On Linux, this would need to be '.amd64-linux'. A more portable solution would substitute '\$SMLNJ\_ARCH' at runtime, but this has not been implemented.

**SML/NJ JSON library not in apt.** On Ubuntu the SML/NJ packages in apt do not include the JSON library, so SML/NJ has to be built from source to obtain '\$/json-lib.cm'.

None of these issues affected the correctness of the implementation, but together they form most of the deployment story for the tool and explain several entries in the further-work discussion in Chapter 7.

## 5.6 Testing Strategy

Each subsystem has its own test 'assert', runnable independently from the SML/NJ REPL. Keeping the unit tests separate (rather than maintaining a single combined one) allowed debugging to occur effectively and ensured each subsystem was working to how we want it to before integration tests occurred.

## 5.7 Test Results

### Unit testing

Each unit test calls the function under test with a known input, handwrites the expected output, and uses SML's structural equality to check whether the actual result matches. Failures are reported with the test name. This pattern was adopted in preference to a full testing framework because Standard ML's structural equality already provides the comparison primitive, and adding a framework would have introduced a dependency for no real benefit at this project's scale.

All 3 automated unit tests pass at the time of writing.

The **parser test** ('mustache\_parser/tests.sml') covers 21 cases including simple variables, unescaped variables, sections, inverted sections, nesting at multiple depths, and the newline stripping rules.

The **preprocessor test** ('preprocessing/tests\_preprocess.sml') covers 13 cases including null-

field handling (both single and multiple fields in the same object), array position flags (single element, two elements, many elements, empty array), and recursive traversal into nested structures.

The **renderer test** (`'renderer/tests_renderer.sml'`) covers 13 cases, including HTML-escaping of the five significant characters, section iteration over arrays, dotted-path lookup, the implicit iterator, inverted sections, and edge cases such as missing keys and empty contexts

### 5.7.1 Specification testing

To validate the renderer against the Mustache standard, I used an automated test suite based on the official Mustache specification. These specifications are a standardised set of YAML files, maintained by the Mustache project, which define exactly what a correct implementation should produce for any given template and dataset. I used a shell script to convert these files to JSON at run time, allowing me to feed each test case through my full pipeline—first the parser, then the renderer, to compare the actual results against the expected output defined in the spec.

In the initial runs, the system passed 91 out of 136 test cases. I performed a manual audit of the 45 failures and confirmed they were not logic errors or bugs; rather, they were tests for advanced features that were deliberately excluded from this project's scope, such as partials, delimiter changes, standalone whitespace stripping, and parent context lookup. Once the test suite was pruned to reflect the features I actually implemented, all remaining tests passed. This successfully demonstrates that the renderer correctly conforms to the official Mustache specification for every feature it supports.

### Integration testing

To test the project itself, I embedded integration testing via the command line interface, which is specified in my `renderer.md` in [62]. The process involved executing the renderer with the command:

```
./render template.mustache data.json
```

These tests confirmed that the components work together correctly and that the enriched JSON produced by the preprocessor is directly consumable by the renderer without failures. To evaluate the project's practicality and scalability, I utilised a range of hand-written datasets, including a 27 line `small_test`, a 383 line `animals` file, and a 567 line `user_dataclass` scenario. The rendered outputs can be viewed in the appendix,

## 5.8 Limitations Confirmed by Testing

Two known gaps surfaced through testing. Partial and delimiters changes are both parsed correctly into the AST, but ignored by the renderer. The parser tests confirm that `'{{>file}}'` produces a 'Partial' node, and that `'{{=« »=}}'` is recognised as a delimiter change, but the renderer tests confirm that both currently render as empty strings. These gaps were intentional, as discussed in Chapter 3 and revisited in Chapter 7, but they are real limits on the prototype's coverage of the Mustache syntax.

## 6. Results and Discussion

This chapter presents the outcomes of the project and evaluates the approach against the original aims, compares the typed preprocessing pipeline against manual preprocessing, and identifies the directions for further work that the project has surfaced.

### 6.1 Findings and Systems Evaluation

This section aims to outline the primary outcomes of the project.

#### 6.1.1 Functional Prototype

The project delivered a complete preprocessing and rendering pipeline. The final tool is a command-line utility that transforms raw JSON data into an enriched format specifically tailored for Mustache templates. The implementation consists of three decoupled subsystems: a parser handling all eight Mustache node types, a recursive preprocessor, and an AST-based renderer.

#### 6.1.2 Testing Evaluation

The system was evaluated through multiple rounds of testing:

1. **Unit Tests:** All 47 internal unit tests pass, confirming the logic of the parser, preprocessor, and renderer. This showcases how each individual subsystem works independently.
2. **Integration Tests:** Beyond simple unit tests, the system was stress-tested against larger datasets, including a 567-line `user_dataclass` file and a 383-line `animals` file. These tests confirmed that the preprocessor's recursive logic handles nesting and volume without performance degradation. This showcased how they are able to work together for the purpose of this project.
3. **Official Specification Testing:** To validate conformance, the renderer was tested against the official Mustache specification suite. Of the 136 test cases, 91 passed initially. A manual audit confirmed that the remaining 45 "failures" were due to features deliberately excluded from the project scope (e.g., `partials` and `delimiter` changes). Once the suite was pruned to match the implemented feature set, the system achieved a 100% pass rate, proving the core engine is strictly compliant with the Mustache standard.

## 6.2 Evaluation of Project Goals

The following addresses the evaluation of the project against the original objectives of the project.

- **Compile time correctness (Achieved)** The central thesis of the project was that SML's type system could automate preprocessing safely. This was fully achieved through exhaustive pattern matching. The SML compiler effectively acted as a formal verification tool, catching omissions during development that would have resulted in silent runtime failures in dynamic languages.
- **Data Preservation and Determinism (Achieved)** A core requirement was that enrichment be purely additive. The project successfully implemented a "JSON-in, JSON-out" flow that preserves original data values and ordering. Furthermore, by avoiding non-deterministic structures like arbitrary map iteration, identical inputs universally produce identical outputs, satisfying the requirement for deterministic rendering.
- **Separation of Concerns (Achieved)** The architecture showcased individual subsystems of the Parser, Preprocessor, and Renderer. This separation allowed for independent testing and proved that the preprocessor is engine-agnostic; it could theoretically sit in front of any Mustache-compliant renderer.
- **Full Mustache Specification (Partial)** While the core logic-less features were fully implemented, "Partials" and "Delimiter changes" were excluded.

## 6.3 Comparative Analysis and Further work

Comparing the manual approach to the SML preprocessor reveals a significant reduction in both developer labour. In a standard test case involving a nullable field and a five-item array, a developer would typically need to write twelve manual data augmentations, such as injecting `isFirst` and `isLast` flags. The SML preprocessor reduces this effort to zero. While manual preparation is brittle and prone to silent failures like stray commas or missing flags, the SML implementation uses recursive, exhaustive traversal to make such omissions structurally impossible, ensuring that new data fields are handled automatically without any manual code updates.

Furthermore, the automated approach offers a massive advantage in maintainability and code efficiency. Manual preprocessing logic tends to scale linearly with data complexity, often swelling into hundreds of lines of bespoke, fragile code that must be rewritten as the data schema evolves. In contrast, the SML preprocessor remains a constant 50 lines regardless of nesting depth or array size. By shifting the burden of data preparation from the developer to a type-checked pipeline, the project successfully eliminates an entire category of rendering

errors while keeping the codebase significantly more compact and robust.

A few reflections on this project:

- Implementation effort was uneven. The parser generated the most code but was mechanically simple via CM-Lex/CM-Yacc. The core contribution, the preprocessor, was remarkably compact due to SML's pattern matching. Consequently, most development time was spent on the renderer and navigating build to suit the environment.
- Standard ML proved well-suited for implementing the renderer and including contextual information. Its type system catches errors at compile time, meaning many mistakes are flagged before the program even runs. However, setting it up in practice was more difficult than expected. The JSON library is not included in the standard SML/NJ installation and must be built from source manually. Additionally, the compiled output is named differently depending on the platform, for example `.amd64-darwin` on macOS and `.amd64-linux` on Linux, meaning the program breaks when moved between machines. These practical issues would make this approach difficult to adopt outside of a controlled development environment. For further progress on this topic, this will need to be mitigated.
- The 47 passing tests and 100% scoped spec compliance are strong evidence of functional correctness within scope. However, because they only test anticipated data shapes, this does not reflect reality, thus further unpredictable testing should be experimented with.

## 6.4 Further work

The project surfaces several clear directions for future engineering and research:

- To become production-ready, the system must be stress-tested against pathological JSON shapes, extreme recursion depths to identify limits, and unusual Unicode characters.
- Partial and delimiter changes, which were excluded to focus on the type-safety argument, could be cleanly implemented via a separate pre-loading pass and lexer state-threading.
- Developing an architecture-aware build script and a Windows port would mitigate SML's ecosystem friction and lower the barrier to entry.
- Because the preprocessor operates strictly on the JSON datatype, it is engine-agnostic. The pipeline could easily be adapted for other logic-less engines, such as Liquid, by swapping the final rendering module.



## 7. Conclusions

This dissertation set out to investigate whether Standard ML’s type system could automate the preprocessing burden imposed by logic-less template systems, with Mustache as the working example. The preceding chapters have described the literature that motivated the project (reduce preprocessing burden), the requirements derived from that literature, the design decisions behind a prototype implementation, the implementation itself, and the results of testing it.

The core contribution is a working prototype: a Mustache parser, a JSON preprocessor with two type-checked transformations, and a renderer, composed by a single command-line entry point. The preprocessor sits in front of any standard Mustache implementation, preserves the cross-language portability that makes Mustache useful, and uses pattern-matching exhaustiveness to ensure that the transformations are applied consistently across nested data structures. All 47 unit tests pass within the scope they were designed to cover, and end-to-end integration tests confirm the components compose correctly but are strictly limited to the tests.

The contribution is not a production-ready tool. It is evidence that the typed-functional approach to preprocessing automation is viable for the cases it was designed for, and that the architecture generalises naturally to additional transformations. Two parts of the Mustache specification (partials and delimiter changes) are parsed but not rendered. The deployment story has rough edges that would matter at scale: the build script hard-codes a platform-specific heap image suffix, several development paths are absolute rather than relative, and the SML/NJ JSON library has to be built from source on Ubuntu. The testing is limited in variety and exercises the cases the preprocessor was designed for; it does not stress-test the system against unusual JSON shapes that a real-world deployment would encounter.

However, the point of a prototype is to demonstrate that this project has some successes, especially as an initial prototype, it shows three things. First, that a typed functional language can express the transformations that logic-less templates require, in code whose size is a constant of the language rather than a function of the data. Second, that pattern-matching exhaustiveness catches errors of omission at compile time rather than at render time, which is the property the literature identified as missing from existing systems. Third, that the preprocessor architecture decouples cleanly from any particular Mustache implementation, which means the approach is in principle deployable in any of the dozens of language ecosystems that already use Mustache.

The most significant directions for further work are widening the transformation library be-

yond the two patterns implemented here, testing the system against a different real-world JSON to identify failure modes the handwritten test data did not exercise, and quantifying the reduction in developer effort that the typed approach provides. Each of these would extend the work from a proof of concept toward a demonstration in production.

The wider implication, if the approach generalises, is that the choice between security and usability that Parr (2004) described as fundamental to template-system design is not actually as binary as the literature has presented it. Mustache makes the secure choice and pays in preprocessing work; logic-friendly engines make the usable choice and pay in security vulnerabilities. The prototype here suggests that the cost of the secure choice can be substantially reduced by moving the preprocessing into a typed functional language where the compiler does the work of catching omissions. Whether that suggestion holds up at production scale is a question for further work, but the prototype shows it is worth investigating into.

## **A. Appendix**

## **B. Mustache Template Example: Student Profile Generation**

All logic must be pre-computed in the data structure, and the template merely renders what is provided. This separation ensures a clear boundary between presentation and business logic. Consequently, there is no string concatenation, arithmetic, or function calls are possible within the template and all formatting decisions (eg, commas or comments) require pre-processing.

### **Input Data**

The JSON data structure contains all pre-computed values, including boolean flags that determine which sections to render, sometimes the tags may miss certain pre-computed values. This necessitates a good .json:

*Listing B.1: Input data (data.json)*

```
{
  "vars": {
    "people": [
      {
        "name": "JohnSmith",
        "email": "john@example.com",
        "degree": "BSc Computer Science",
        "isComputerScience": true,
        "skills": [
          {"skill": "Python"},
          {"skill": "JavaScript"},
          {"skill": "SQL", "last": true}
        ]
      }
    ]
  }
}
```

#### Template Structure

*Listing B.2: Mustache template (example.mustache)*

```
{{#vars}}                                // Section: only renders if vars exists
{{#people}}                               // Iteration: renders once per person
{{#isComputerScience}}                   // Conditional: renders if true
// Computer Science Student              // No if/else logic - just presence check
public class {{name}}ComputerScienceProfile
{
    public string Name = "{{name}}";      // Simple substitution
    public string Email = "{{email}}";    // No string manipulation
    public string Degree = "{{degree}}";  // No computation

    public bool IsComputerScience = true;
    public string Specialization = "Software_Development";
{{/isComputerScience}}
{{^isComputerScience}}                   // Inverted: renders if false
// Non-Computer Science Student         // Still no logic - just absence check
public class {{name}}NonComputerScienceProfile
{
    public string Name = "{{name}}";
```

```

    public string Email = "{{email}}";
    public string Degree = "{{degree}}";
{{/isComputerScience}}

    public string[] Skills = { {{#skills}}"{{skill}}"{{^last}},
    {{/last}}{{/skills}} };
}
// Array iteration with separator

{{/people}}
{{/vars}}

```

### Generated Output

The rendered output shows how the template collects the desired information based on the provided data:

#### *Listing B.3: Generated C# output*

```

// Computer Science Student
public class JohnSmithComputerScienceProfile
{
    public string Name = "JohnSmith";
    public string Email = "john@example.com";
    public string Degree = "BSc_Computer_Science";

    public bool IsComputerScience = true;
    public string Specialization = "Software_Development";

    public string[] Skills = { "Python", "JavaScript", "SQL" };
}

```

### Mustache Implementation Languages

Mustache is implemented in over 50 languages: Ruby, JavaScript, Python, Erlang, Elixir, PHP, Perl, Raku, Objective-C, Java, C/.NET, Android, C++, CFEngine, Go, Lua, ooc, ActionScript, ColdFusion, Scala, Clojure[Script], Clojure, Fantom, CoffeeScript, D, Haskell, XQuery, ASP, Io, Dart, Haxe, Delphi, Racket, Rust, OCaml, Swift, Bash, Julia, R, Crystal, Common Lisp, Nim, Pharo, Tcl, C, ABAP, Elm, Kotlin, SQL, PowerShell, Odin, and Arduino [16]

## C. Rendered Output

### C.1 "small\_test" rendered output

```
People:
- Alice (Computer Science)
  Email: alice@example.com
- Bob (Mathematics)
  Email: bob@example.com
```

### C.2 "animals" rendered output snippet

```
from dataclasses import dataclass
from typing import Optional, List

@dataclass
class SnowLeopard:
    """
    The snow leopard is a large cat native to the
    mountain ranges of Central and South Asia,
    renowned for its elusive nature and stunning coat.
    // "injected contextual description"

    Scientific: Panthera uncia
    Class:      Mammalia
    Diet:       carnivore
    Habitat:    montane
    Status:     Vulnerable
    Fun fact:   Snow leopards cannot roar -- they
    communicate with chuffs, mews and growls.
    """

    id: str = "sp-001"
    common_name: str = "Snow Leopard"
    scientific_name: str = "Panthera uncia"

    taxonomy_kingdom: str = "Animalia"
    taxonomy_phylum: str = "Chordata"
    taxonomy_class: str = "Mammalia"
```

```

taxonomy_order: str = "Carnivora"
taxonomy_family: str = "Felidae"
taxonomy_genus: str = "Panthera"
taxonomy_species: str = "uncia"

conservation_status: str = "VU"
conservation_status_full: str = "Vulnerable"
conservation_population_estimate: Optional[int] = 4000
conservation_population_trend: str = "decreasing"

weight_kg_min: float = 22
weight_kg_max: float = 55
length_cm_min: float = 75
length_cm_max: float = 130
lifespan_wild: int = 10
lifespan_captivity: int = 21

habitat_biome: str = "montane"
habitat_description: str = "Rocky mountain terrain,
alpine meadows and steep cliffs."
altitude_min: int = 3000
altitude_max: int = 5500

diet: str = "carnivore"
activity: str = "crepuscular"
social: str = "solitary"
territorial: bool = True
migration: bool = False

gestation_days: Optional[int] = 93
litter_size_min: int = 1
litter_size_max: int = 5
breeding_season: str = "January to March"
sexual_maturity_years: int = 2

image_url: str = "https://example.com/images/snow_leopard.jpg"

threats: List[str] = (
    ["habitat loss", "poaching", "prey decline",
     "climate change"]
)

```



```

countries: List[str] = (
    ["Afghanistan", "Bhutan", "China", "India",
     "Mongolia", "Nepal", "Pakistan", "Russia"]
)

prey: List[str] = (
    ["blue sheep", "ibex", "marmots", "birds"]
)

tags: List[str] = (
    ["big cat", "endangered", "mountain", "solitary"]
)

related_species: List[str] = (
    ["sp-002"]
)
...

```

### C.3 "users\_dataclass" rendered output snippet

```

from dataclasses import dataclass
from typing import Optional, List

@dataclass
class alice_wonder:
    """
    Software engineer and avid reader. // "injected contextual
    description"

    Role:      admin
    Location:  London, UK
    Twitter:   @alice_wonder
    GitHub:    alice-wonder
    LinkedIn:  alice-wonderland
    """

    username: str = "alice_wonder"
    email: str = "alice@example.com"

```

```

full_name: str = "Alice Wonderland"
age: int = 28
role: str = "admin"
active: bool = True
verified: bool = True

tags: List[str] = (
    ["power-user", "beta-tester", "contributor"]
)

def __repr__(self):
    return f"{self.username}(username={self.username!r},
        role={self.role!r}, active={self.active})"

def is_admin(self):
    return self.role == "admin"

def is_active(self):
    return self.active


@dataclass
class bob_builder:
    """
    Hobbyist woodworker and DIY enthusiast.

    Role:      user
    Location: Manchester, UK
    GitHub:    bobbuidls
    """

    username: str = "bob_builder"
    email: str = "bob@example.com"
    full_name: str = "Bob Builder"
    age: int = 35
    role: str = "user"
    active: bool = True
    verified: bool = False

    tags: List[str] = (
        ["new-user"]
    )

```

```

)

def __repr__(self):
    return f"{self.username} (username={self.username!r},
        role={self.role!r}, active={self.active})"

def is_admin(self):
    return self.role == "admin"

def is_active(self):
    return self.active

@dataclass
class carol_danvers:
    """
    Community manager and cat lover.

    Role:      moderator
    Location:   Edinburgh, UK
    Twitter:    @carol_d
    LinkedIn:   carol-danvers-mod
    """

    username: str = "carol_danvers"
    email: str = "carol@example.com"
    full_name: str = "Carol Danvers"
    age: int = 31
    role: str = "moderator"
    active: bool = False
    verified: bool = True

    tags: List[str] = (
        ["moderator", "long-term"]
    )

    def __repr__(self):
        return f"{self.username} (username={self.username!r},
            role={self.role!r}, active={self.active})"

    def is_admin(self):

```

```
        return self.role == "admin"

    def is_active(self):
        return self.active
```

D. Project Management

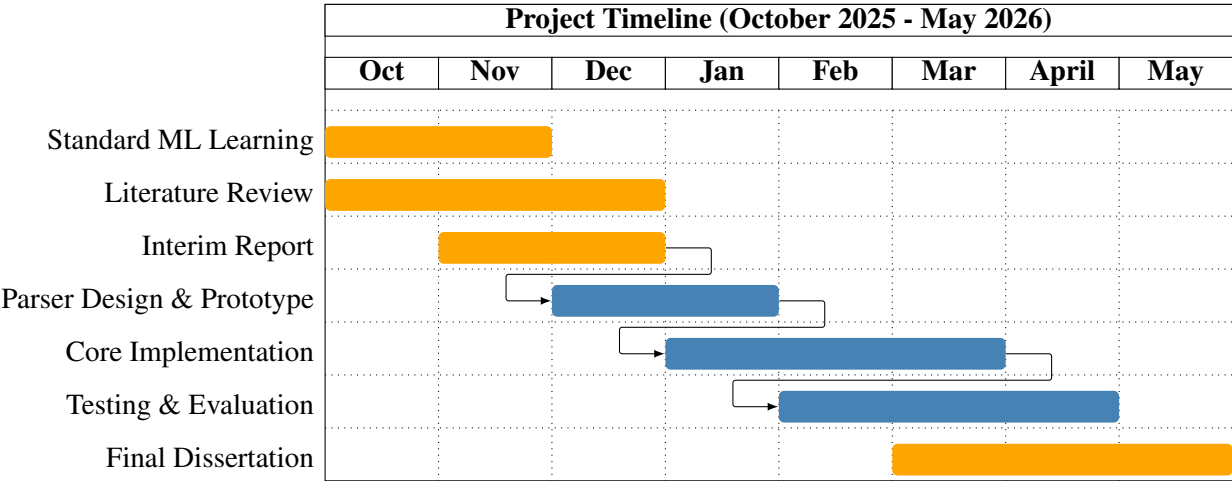


Figure D.1: Gantt chart showing major phases from October 2025 to May 2026. Orange bars indicate research and writing phases; blue bars indicate implementation work. Detailed weekly breakdown provided in Sections 2.3 and 2.4. The critical path runs through pattern matching learning (Feb) and core implementation (Feb-Mar).

The project followed an iterative, incremental approach, with the system built in distinct stages, lexer, parser, preprocessor, and renderer: each tested and integrated before the next began. Version control was managed with Git, and the repository was hosted on GitHub ([github.com/jylew1/Templating-in-SML](https://github.com/jylew1/Templating-in-SML) [62]). Weekly supervisory meetings with my supervisor were important to keeping the project on track, providing a regular checkpoint to discuss progress and challenges throughout the week, and shaping key decisions such as the scope of the literature review and the framing of the contribution.

Time was managed through a steady routine (about an hour each weekday and an hour and a half on weekend days). This suited the project and ensured tasks were completed daily, to which consistency aided largely.

## E. Self-review

Coming from a Biomedical Engineering background, the gap in computer science knowledge was a challenge, early on in the project. Standard ML concepts like as signatures, modules, and algebraic datatypes required substantial background reading before I could implement it myself, thus progress in the first weeks was correspondingly slower than expected. After a while, I experienced challenges, yet more productive progress was made, in particular during my implementation stage D.1. In hindsight, this became a very rewarding learning experience: not only did I learn a lot about functional languages and templating engines, but also a lot about computer science.

My development as a writer and a programmer was driven largely by my supervisor's feedback, which repeatedly guided me in directions I didn't know myself until much later. I found that revisiting sections after a few days consistently revealed ways to sharpen the argument and improve clarity, and multiple revision passes became standard practice rather than the exception. The project became more interesting as the literature review developed: working through Parr's model-view separation and Pisu's security analysis transformed what began as a vague aim of automating preprocessing from my personal experience of templating languages. This progressed into a focused investigation of whether type systems can resolve the security.

Another lesson involved my time management. I underestimated how long the early fundamentals would take, and even after that, it compressed the implementation and write-up stages more than I would have expected. In future projects, I would build in significantly more buffer time, particularly where progress depends on learning unfamiliar material before any real work can begin. Overall the project met its aims: improving in my literature review, ability to produce technical work and deliver a working pipeline alongside a clear understanding of the problem it addresses.

## Bibliography

- [1] Helm Contributors, *Helm: The package manager for Kubernetes*, Official Website. [Online]. Available: <https://helm.sh/>.
- [2] F. Y. Rashid, *Infrastructure as code templates are the source of many cloud infrastructure weaknesses*, CSO Online, Jan. 2024. [Online]. Available: <https://www.csoononline.com/article/568909/infrastructure-as-code-templates-are-the-source-of-many-cloud-infrastructure-weaknesses.html>.
- [3] Pallets, *Jinja*, Official Website, 2008. [Online]. Available: <https://palletsprojects.com/p/jinja/>.
- [4] Wikipedia contributors, *FreeMarker*, Wikipedia, The Free Encyclopedia, 2024. [Online]. Available: <https://en.wikipedia.org/wiki/FreeMarker>.
- [5] L. Pisu, D. Maiorca, and G. Giacinto, “An assessment of the overlooked dangers of template engines,” *arXiv preprint arXiv:2405.01118*, Mar. 2025. DOI: <https://doi.org/10.1145/3799796>. arXiv: 2405.01118 [cs.CR]. [Online]. Available: <https://dl.acm.org/doi/pdf/10.1145/3799796>.
- [6] PortSwigger, *Server-side template injection*, Web Security Academy, Accessed: 12 May 2026, 2015. [Online]. Available: <https://portswigger.net/web-security/server-side-template-injection>.
- [7] OWASP Foundation, *OWASP Top 10: 2021*, Open Web Application Security Project, 2021. [Online]. Available: <https://owasp.org/Top10/>.
- [8] Evalian, *OWASP top 10 security vulnerabilities 2021*, Evalian Blog, Oct. 2021. [Online]. Available: <https://evalian.co.uk/owasp-top-10-security-vulnerabilities-2021/>.
- [9] J. Kettle, *Server-side template injection*, PortSwigger Research, Whitepaper accompanying Black Hat USA 2015 presentation, Aug. 2015. [Online]. Available: <https://portswigger.net/research/server-side-template-injection>.
- [10] O. Tsai, *uber.com may RCE by Flask Jinja2 Template Injection*, HackerOne Report #125980, Mar. 2016. [Online]. Available: <https://hackerone.com/reports/125980>.
- [11] A. Tiurin, *Exploiting SSTI in Thymeleaf*, Acunetix Web Security Blog, Jun. 2020. [Online]. Available: <https://www.acunetix.com/blog/web-security-zone/exploiting-ssti-in-thymeleaf/>.
- [12] Y. Zhao, Y. Zhang, and M. Yang, “Remote code execution from SSTI in the sandbox: Automatically detecting and exploiting template escape bugs,” in *Proc. 32nd USENIX*



- Security Symposium (USENIX Security 23)*, USENIX Association, 2023, pp. 3691–3708.
- [13] T. J. Parr, “Enforcing strict model-view separation in template engines,” in *Proceedings of the 13th International Conference on World Wide Web*, ser. WWW ’04, New York, NY, USA: ACM, May 2004, pp. 224–233, ISBN: 1-58113-844-X. DOI: 10.1145/988672.988703.
  - [14] *Mustache manual*, Mustache, Sep. 2022. [Online]. Available: <https://mustache.github.io/mustache.5.html>.
  - [15] Mustache Organization, *Mustache github repository*, GitHub, 2009. [Online]. Available: <https://github.com/mustache/mustache>.
  - [16] Mustache, *Mustache - logic-less templates*, Official Website, 2009. [Online]. Available: <https://mustache.github.io/>.
  - [17] S. Pottavathini, *The case against logic-less templates*, eBay Inc. Engineering Blog, Oct. 2012. [Online]. Available: <https://innovation.ebayinc.com/stories/the-case-against-logic-less-templates/>.
  - [18] A. Boronine, *Cult of logic-less templates*, Alexei Boronine’s Blog, Sep. 2012. Accessed: Sep. 7, 2012. [Online]. Available: <https://www.boronine.com/2012/09/07/Cult-Of-Logic-less-Templates/>.
  - [19] S. Pullara, *Mustache is logic-less but the logic has to go somewhere*, Medium, Jun. 2016. [Online]. Available: <https://medium.com/spullara/mustache-is-logic-less-but-the-logic-has-to-go-somewhere-3e976b7a49c8>.
  - [20] M. Samuel, P. Saxena, and D. Song, “Context-sensitive auto-sanitization in web templating languages using type qualifiers,” in *Proceedings of the 18th ACM Conference on Computer and Communications Security (CCS ’11)*, Chicago, Illinois, USA: ACM, Oct. 2011, pp. 587–600. DOI: 10.1145/2046707.2046775.
  - [21] A. Agarwal, S. Mondet, P. Veber, C. Troestler, and F. Berenger, “Biocaml: The OCaml bioinformatics library,” in *OCaml Users and Developers Meeting (OUD)*, Copenhagen, Denmark, Sep. 2012. [Online]. Available: <http://biocaml.org/>.
  - [22] F. Berenger, K. Y. J. Zhang, and Y. Yamanishi, “Chemoinformatics and structural bioinformatics in OCaml,” *Journal of Cheminformatics*, vol. 11, no. 1, p. 24, Feb. 2019. DOI: 10.1186/s13321-019-0332-0. [Online]. Available: <https://jcheminf.biomedcentral.com/articles/10.1186/s13321-019-0332-0>.
  - [23] Wikipedia contributors, *Smalltalk Wikipedia, the free encyclopedia*, <https://en.wikipedia.org/wiki/Smalltalk>, [Online; accessed 14-May-2026], 2026.
  - [24] G. E. Krasner and S. T. Pope, “A cookbook for using the model-view-controller user interface paradigm in Smalltalk-80,” *Journal of Object-Oriented Programming*, vol. 1, no. 3, pp. 26–49, Aug. 1988.

- [25] K. Gao, Y. Sun, and C.-A. Sun, *Understanding bugs in template engine-based applications: Symptoms, root causes, and fix patterns*, 2018. arXiv: 2604 . 27692 [cs.SE]. [Online]. Available: <https://arxiv.org/abs/2604.27692>.
- [26] Wikipedia contributors, *Mustache (template system)*, Wikipedia, The Free Encyclopedia, 2024. [Online]. Available: [https://en.wikipedia.org/wiki/Mustache\\_\(template\\_system\)](https://en.wikipedia.org/wiki/Mustache_(template_system)).
- [27] J-Gallo. “Mustache vs Handlebars.”[Online]. Available: <https://medium.com/@JuaniGallo/mustache-vs-handlebars-42e531eca252>.
- [28] Shopify, *Liquid: Safe, customer-facing template language for flexible web apps*, <https://shopify.github.io/liquid/>, 2024.
- [29] F. Potencier, *Twig: The flexible, fast, and secure template engine for php*, <https://twig.symfony.com>, 2024.
- [30] Mozilla, *Nunjucks: A powerful templating engine with inheritance*, GitHub, 2012. [Online]. Available: <https://mozilla.github.io/nunjucks/>.
- [31] GitHub Advisory Database, *CVE-2019-10906: Jinja2 sandbox escape via string formatting*, <https://github.com/advisories/GHSA-462w-v97r-4m45>, Accessed: [Insert Date], 2019.
- [32] National Vulnerability Database, *CVE-2024-56326: Jinja sandboxed environment indirect call bypass*, <https://nvd.nist.gov/vuln/detail/CVE-2024-56326>, Accessed: [Insert Date], 2024.
- [33] D. Thomas, C. Fowler, and A. Hunt, *Programming Ruby: The Pragmatic Programmers’ Guide*, 4th. Raleigh, NC: Pragmatic Bookshelf, 2013.
- [34] EJS Team, *Ejs – embedded javascript templates*, Official Website, 2010. [Online]. Available: <https://ejs.co/>.
- [35] React Team, *Writing markup with jsx*, React Documentation, 2023. [Online]. Available: <https://react.dev/learn/writing-markup-with-jsx>.
- [36] Svelte Team, *Svelte documentation*, Official Website, 2016. [Online]. Available: <https://svelte.dev/docs>.
- [37] Scalate Project, *Scalate: Scala template engine*, <https://scalate.github.io/scalate/>, Includes Mustache support for the Play Framework, 2024.
- [38] D. VandenBerghe, *The case for ml*, comp.compilers forum, republished on Yale FLINT website, Jul. 1998. [Online]. Available: <https://flint.cs.yale.edu/cs421/case-for-ml.html>.
- [39] Wikipedia contributors, *ML (programming language)*, Wikipedia, The Free Encyclopedia, 2024. [Online]. Available: [https://en.wikipedia.org/wiki/ML\\_\(programming\\_language\)](https://en.wikipedia.org/wiki/ML_(programming_language)).
- [40] S. Peyton Jones, *Haskell 98 Language and Libraries: The Revised Report*. Cambridge: Cambridge University Press, 2003.

- [41] M. Snoyman, *Hamlet: Type-safe templates via Template Haskell*, Yesod Web Framework, 2012. [Online]. Available: <https://www.yesodweb.com/book/shakespearean-templates>.
- [42] J. Van Der Jeugt, *Blazehtml: A blazingly fast html combinator library*, Personal Website, 2010. [Online]. Available: <https://jaspervdj.be/blaze/>.
- [43] X. Leroy, D. Doligez, A. Frisch, J. Garrigue, D. Rémy, and J. Vouillon, *The ocaml system release 4.02: Documentation and user's manual*, Institut National de Recherche en Informatique et en Automatique, 2014.
- [44] T. Bunko, *Jingoo: Template engine for ocaml*, GitHub, 2013. [Online]. Available: <https://github.com/tategakibunko/jingoo>.
- [45] L. Wendel, *Elm-mustache: A mustache implementation for elm*, <https://package.elm-lang.org/packages/lukewestby/elm-string-interpolate/latest/>, Logic-less template rendering in a strictly typed functional language, 2018.
- [46] R. Milner, M. Tofte, R. Harper, and D. MacQueen, *The Definition of Standard ML (Revised)*. Cambridge, MA: MIT Press, 1997.
- [47] A. W. Appel, *Modern Compiler Implementation in ML*. Cambridge, UK: Cambridge University Press, 1998.
- [48] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd. Boston, MA: Pearson Education, 2006.
- [49] Wikipedia contributors, *Chomsky hierarchy — Wikipedia, the free encyclopedia*, [Online; accessed 16-May-2026], 2026. [Online]. Available: [https://en.wikipedia.org/wiki/Chomsky\\_hierarchy](https://en.wikipedia.org/wiki/Chomsky_hierarchy).
- [50] Wikipedia contributors, *Parsing — Wikipedia, the free encyclopedia*, [Online; accessed 16-May-2026], 2026. [Online]. Available: <https://en.wikipedia.org/wiki/Parsing#Lookahead>.
- [51] A. Turon, *SML/NJ language processing tools: User guide*, Standard ML of New Jersey, 2009. [Online]. Available: <https://www.smlnj.org/doc/ml-lpt/manual.pdf>.
- [52] G. Hutton and E. Meijer, “Monadic parser combinators,” *Journal of Functional Programming*, vol. 8, no. 4, pp. 437–444, 1996.
- [53] Wikipedia contributors, *Parser combinator*, Wikipedia, The Free Encyclopedia, [Online; accessed 15-May-2026], 2026. [Online]. Available: [https://en.wikipedia.org/wiki/Parser\\_combinator](https://en.wikipedia.org/wiki/Parser_combinator).
- [54] M. Blume, “The sml/nj compilation manager (cm) user manual,” Lucent Technologies, Bell Labs, Tech. Rep., 2000. [Online]. Available: <https://www.smlnj.org/doc/CM/new.pdf>.
- [55] K. Crary, *Cm-lex and cm-yacc user's manual (version 2.1)*, Carnegie Mellon University, Dec. 2017. [Online]. Available: <https://www.cs.cmu.edu/~crary/cmtool/manual.pdf>.

- [56] Wikipedia contributors. “Comparison of parser generators,” Wikipedia, The Free Encyclopedia, Accessed: May 17, 2026. [Online]. Available: [https://en.wikipedia.org/wiki/Comparison\\_of\\_parser\\_generators](https://en.wikipedia.org/wiki/Comparison_of_parser_generators).
- [57] R. Nystrom, *Crafting Interpreters*. Genever Benning, 2021, ISBN: 978-0990582939.
- [58] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 2nd. Morgan Kaufmann, 2011.
- [59] A. Holovaty and J. Kaplan-Moss, *Mustache: Logic-less templates*, <https://mustache.github.io>, 2009.
- [60] M. Moore and A. Gureiev, *10 compelling reasons to choose python for your next project*, Shakuro Blog, Jan. 2024. [Online]. Available: <https://shakuro.com/blog/ten-reasons-to-choose-python>.
- [61] SML Family, *Standard ml family GitHub project*, GitHub Pages, 2015. [Online]. Available: <https://smlfamily.github.io/>.
- [62] J. Y. Lew, *Templating in Standard ML: Project repository*, GitHub, 2025. [Online]. Available: <https://github.com/jylew1/Templating-in-SML>.